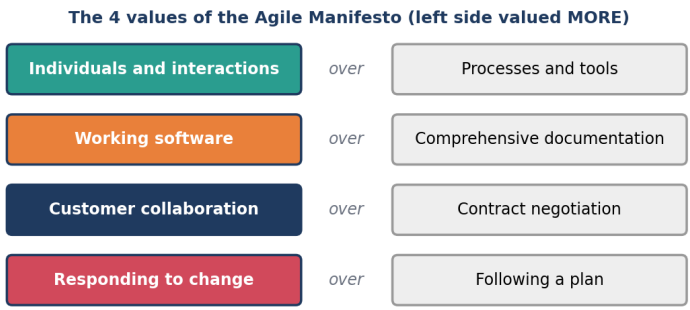
Agile Software Development

Question Bank - Model Answers with Diagrams

Unit 1: Why Agile? and How to Be Agile? (Q1 - Q10)
Unit 2: Agile Manifesto, Design, Testing, Tools and Case Studies (Q1 - Q9)



How to use: Each answer is written exam-style (definition, explanation, example, diagram) so you can reproduce it in your own words within the marks given. Case-study answers follow the sub-parts (a, b, c) exactly as asked. Unit 1 follows *The Art of Agile Development* (James Shore); Unit 2 follows the Agile Manifesto and its 12 principles.
Note: Unit 2 question 10 in the question bank is blank, so it is not answered. In Unit 1, Q5 lists '4 marks' in its stem but the bank marks it as 10, so it is answered for 10.

UNIT 1 - Q1. Understanding Success (10 marks)

RBT L2 (Understand)

Question: Explain the different dimensions of success, and why 'success' for a software project needs a broader definition than 'the software works'.

Introduction

Many teams treat a project as successful when the software runs, and was delivered on time and within budget. Shore argues this is far too narrow. A project can meet every requirement and still be a failure if nobody benefits from it, if it cannot be maintained, or if the team burns out building it. Success therefore has to be judged along several dimensions at the same time.

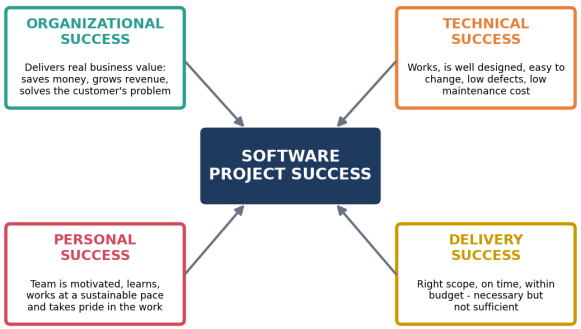


Fig 1.1 - Dimensions of software project success

Dimensions of success

Dimension	What it means	Example / How to check
Organizational success	The software creates real value for the organization that paid for it: saves money, earns money, improves service, or meets a strategic goal.	A bank's mobile app that reduces branch visits by 30%, so the bank saves cost.
Technical success	The software works correctly, has a clean, simple design, few bugs, and can be changed cheaply when needs change.	New features take days, not months; defect count falls release by release.
Personal success	The people building it are motivated, keep learning, work at a sustainable pace and are proud of the result.	Low turnover, no constant overtime, team wants to work together again.
Delivery success (scope, time, cost)	The traditional measure: the agreed features delivered by the agreed date and budget.	Release on 30 June at Rs 20 lakh. Necessary, but not enough alone.

Why 'the software works' is not enough

- **Working is not the same as useful.** Software can match the specification but fail the user's real need (the specification may be wrong or out of date).
- **Value arrives only when the software is used.** If officers keep using spreadsheets, the investment earns nothing.
- **Technical debt makes success short-lived.** A messy design that 'works today' makes every future change slow and expensive.
- **People are part of the system.** A death-march project produces burnout, attrition and loss of knowledge, hurting the next project.
- **Different stakeholders define success differently** - customers, users, sponsors, developers and maintainers - so a single yes/no test cannot capture it.

Conclusion

A successful project is one that is **valuable to the organization, technically sound, sustainable for the people, and delivered reasonably on time and budget**. Agile is designed to optimise all of these together, with the organizational value as the main driver.

UNIT 1 - Q2. Enter Agility (10 marks)

RBT L2 (Understand)

Question: Define agility. Explain how an agile approach responds to changing requirements through short cycles and frequent delivery.

Definition

Agility is the ability of a team or organization to **respond quickly and effectively to change** while continuing to deliver value. In software development, an agile team accepts that requirements, technology and the market will change, and organises its work (planning, design, coding, testing) so that change is cheap rather than painful.

Everyday analogy: A cab driver using GPS re-routes the moment traffic appears. A driver with a printed map planned months earlier keeps going into the jam. Agile teams behave like the GPS driver: they re-plan continually using fresh information.

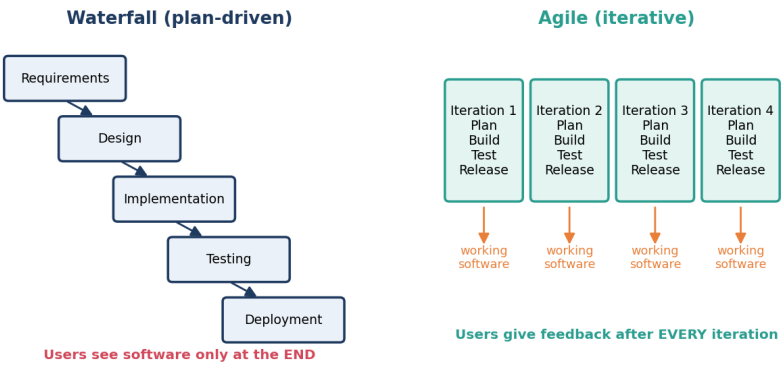


Fig 2.1 - Waterfall gives feedback only at the end; agile gives feedback every iteration

How an agile approach copes with changing requirements

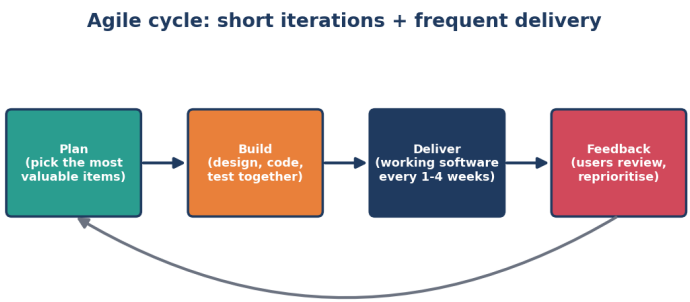


Fig 2.2 - The agile cycle of short iterations and frequent delivery

- **Short cycles (iterations/sprints of 1-4 weeks).** Only a small slice of work is committed at a time, so a change affects at most the next iteration, not a 12-month plan.
- **Frequent delivery of working software.** Users see and try real features every few weeks, so misunderstandings are found early when they are cheap to fix.

- **Continuous prioritisation.** The product backlog is reordered before each iteration; a new, more valuable item (e.g., a new payment option) can jump the queue.
- **Customer collaboration.** The customer is available daily to clarify and adjust, instead of signing off a fixed specification once.
- **Technical practices that make change safe:** automated tests, continuous integration, refactoring and simple design stop the code base from becoming rigid.
- **Retrospectives.** After each iteration the team inspects its process and adapts it.

Example

An online food-ordering team plans a 2-week sprint to build 'search restaurants'. After the demo, users ask for 'filter by veg/non-veg'. In a waterfall project this request would be a change-control battle; in an agile project it simply joins the backlog and is built in the next sprint.

Conclusion

Agility does not mean 'no plan'; it means **planning continuously in small steps**, delivering often, and using feedback to steer. This turns changing requirements from a risk into a competitive advantage.

UNIT 1 - Q3. Agile Methods (10 marks)

RBT L2 (Understand)

Question: Define agile methods, and describe any three well-known agile methods and the main practices each emphasizes.

Definition

Agile methods are lightweight, iterative and collaborative approaches to software development that follow the values of the Agile Manifesto. Each method is a **coherent set of practices, roles and values** that work together to deliver working software in short cycles, with close customer involvement and continuous improvement. Examples: Scrum, Extreme Programming (XP), Kanban, Lean, Crystal, DSDM, FDD.

1. Scrum

Scrum is a **management framework** built around fixed-length sprints (1-4 weeks) in which a cross-functional team produces a potentially releasable increment.

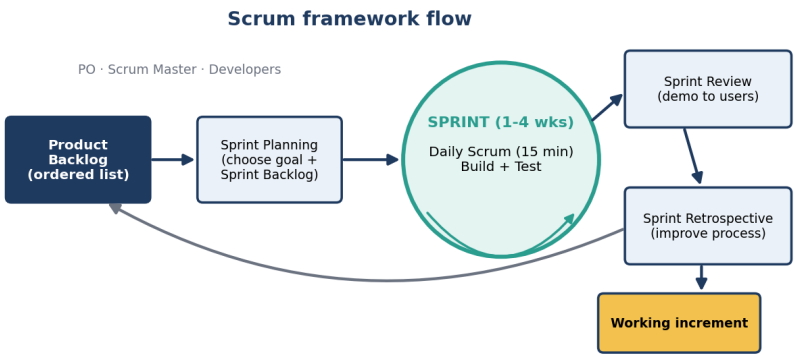


Fig 3.1 - Scrum framework

- **Roles:** Product Owner (decides priorities), Scrum Master (coaches, removes impediments), Developers (self-organizing team).
- **Artifacts:** Product Backlog, Sprint Backlog, Increment.
- **Events:** Sprint Planning, Daily Scrum (15 min), Sprint Review, Sprint Retrospective.
- **Emphasis:** time-boxing, transparency, inspect-and-adapt, prioritised backlog.

2. Extreme Programming (XP)

XP is an **engineering-focused** method that takes good development practices 'to the extreme' to produce high-quality code under changing requirements.

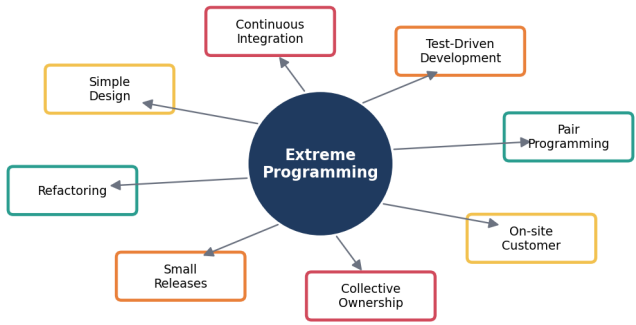


Fig 3.2 - Key practices of Extreme Programming

- **Pair programming:** two people write code together; continuous review.
- **Test-driven development (TDD):** write a failing test first, then the code.
- **Continuous integration:** integrate and test many times a day.
- **Simple design and refactoring, small releases, collective code ownership, on-site customer, 40-hour (sustainable) week.**

3. Kanban

Kanban is a **flow-based** method that visualises work on a board and limits work in progress (WIP) to expose bottlenecks. There are no fixed sprints; work is pulled when capacity is free.

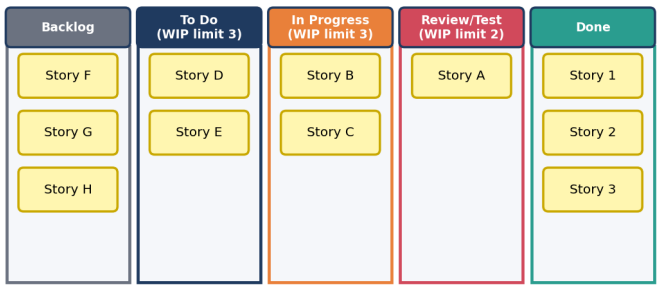


Fig 3.3 - Kanban board with WIP limits

- **Visualise the workflow** (columns such as To Do, In Progress, Review, Done).
- **Limit WIP** so the team finishes work before starting new work.
- **Manage flow** using lead time and cycle time; continuously improve.

Comparison

Aspect	Scrum	XP	Kanban
Focus	Project management	Engineering practices	Workflow / flow efficiency
Cadence	Fixed sprints	Short iterations (1-2 weeks)	Continuous, no sprints
Roles	PO, Scrum Master, Developers	Coach, customer, programmers	No prescribed roles
Key control	Time-box and sprint goal	Tests, pairing, CI	WIP limits

UNIT 1 - Q4. The Road to Mastery (10 marks)

RBT L3 (Apply)

Question: Illustrate, with examples, the stages a practitioner passes through on the road to mastery.

Introduction

Learning agile, like learning a martial art or cooking, happens in stages. Shore uses the Japanese concept **Shu-Ha-Ri** (originating in Aikido) to describe how a practitioner moves from following rules, to bending them, to creating own approach. Knowing your stage tells you how to learn and what to expect from yourself.

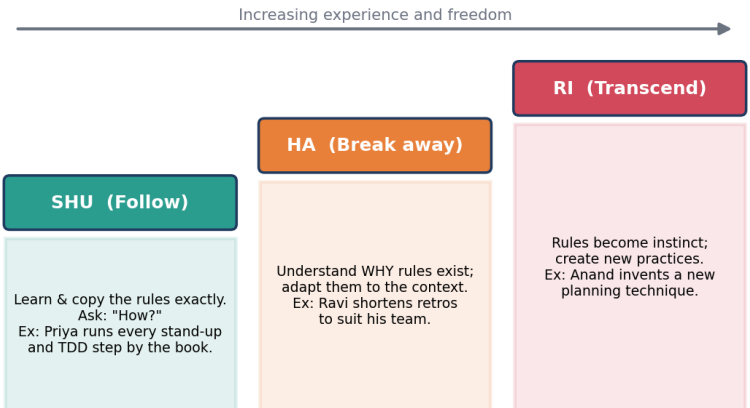


Fig 4.1 - Shu-Ha-Ri: the road to mastery

Stage 1 - Shu (Follow / Learn)

- **Characteristics:** The beginner follows rules exactly as taught, without modification. Concentrates on *how*. Needs concrete, step-by-step guidance and cannot yet judge which rules matter.
- **Example:** A new developer practises TDD strictly: write test -> see it fail -> write minimal code -> refactor. A new team runs Scrum exactly as the guide says - 15-minute daily stand-up, 2-week sprints, retrospective at the end of every sprint - even if it feels awkward.
- **Risk:** Mechanical following without understanding; mistaking the rules for the goal.

Stage 2 - Ha (Break away / Understand)

- **Characteristics:** The practitioner has mastered the basics and now understands *why* each rule exists, so can recognise when a rule does not fit and adapt it. Experiments with variations and combines ideas from different methods.
- **Example:** A team of 3 co-located people finds the daily stand-up redundant because they talk constantly; they switch to a short on-demand sync but keep the purpose (shared awareness). Another team moves from 2-week to 1-week sprints because feedback was too slow.
- **Risk:** Dropping a practice because it is hard, not because it is unnecessary.

Stage 3 - Ri (Transcend / Innovate)

- **Characteristics:** The expert has internalised the principles; practices are natural and no longer thought of as 'rules'. They create new practices that fit unique situations and teach others.
- **Example:** An experienced coach designs a custom planning approach for a distributed hardware+software team, and writes about it so the community can learn from it.
- **Note:** Even masters keep learning; the road has no final end.

Summary table

Stage	Mindset	Relationship to rules	Learning approach
Shu	'Tell me what to do'	Follow exactly	Practise by the book, get a mentor
Ha	'Why does it work?'	Adapt consciously	Experiment, reflect, compare methods
Ri	'What does this situation need?'	Create new	Innovate, mentor others

Conclusion: Progress is not a straight line; people may be at Shu in one skill (e.g., TDD) and at Ha in another (e.g., planning). Honestly identifying your stage prevents both over-rigid and over-confident behaviour.

UNIT 1 - Q5. Find a Mentor (10 marks)

RBT L5 (Evaluate)

Question: Explain the role a mentor plays in helping a team learn agile; discuss the value of an experienced mentor compared with learning only from books and online resources; state the qualities to look for in a mentor.

Role of a mentor

A mentor is an **experienced agile practitioner who coaches the team** as it adopts agile, often by working alongside them for several iterations. The team is a beginner at the Shu stage and a mentor shortens the road to mastery.

- **Teaches by doing:** demonstrates practices (e.g., pairing, TDD, planning games) in the team's actual code and context.
- **Gives timely feedback:** notices problems (skipped tests, bloated stories, silent retrospectives) and corrects them early.
- **Guides adaptation:** explains when a practice can be relaxed (Ha) and when it is critical, protecting the team from dropping the wrong practices.
- **Provides perspective and credibility:** has seen similar projects, can resolve disputes and help persuade management.
- **Builds independence:** the aim is to make the team self-sufficient, then step back.



Fig 5.1 - How a mentor bridges the gap between knowledge and practice

Mentor vs learning only from books and online resources

Criterion	Books / online resources	Experienced mentor
Fit to context	Generic; assumes an average team	Tailored to your team, product and culture
Feedback	None - you may never know you are doing it wrong	Immediate, specific, personal
Tacit knowledge	Hard to convey (judgment, timing, 'feel')	Shown through demonstration and stories
Problem solving	You must search and interpret alone	Diagnoses root causes and suggests options
Motivation / change	Easy to put aside	Accountability and encouragement keep adoption going
Cost / scale	Cheap, always available, wide coverage	Costly, limited availability

Evaluation: Books and online material are excellent for vocabulary and background, and are cheap, but they cannot observe your team. A mentor is most valuable early on and when the team is stuck; the best approach is to **combine both** - read to understand principles and use the mentor to apply them correctly.

Qualities to look for in a mentor

- **Real hands-on experience** on successful agile projects, including the practice in question (not only certification).
- **Good teaching and communication skills**; patient, explains the 'why'.
- **Pragmatic, not dogmatic:** adapts to your situation while keeping the principles.

- **Honest and willing to challenge** the team and management constructively.
- **Humility and continuous learning;** admits mistakes and shares them.
- **Focus on making the team independent,** not creating dependency.
- **Compatible personality and values**; earns the team's trust.

UNIT 1 - Q6. Case Study: HealthBridge (10 marks)

Case: HealthBridge chose the plan that promised the lowest development cost. The delivered system met every specified requirement but did not reduce patient waiting time (the hospital's real goal). The hospital refused to renew the contract.

(a) Why meeting requirements and budget did not give organizational success (4 marks)

- **Requirements were a proxy, not the goal.** The hospital's real objective was shorter waiting time. The requirements document described features, not outcomes, so a system could satisfy every line and still not improve waiting.
- **Success was measured by cost and conformance.** Choosing the cheapest plan optimised the vendor's cost, not the hospital's value. The cheapest option usually cuts analysis, user involvement and testing - the activities that discover real needs.
- **No feedback loop with real users.** Staff such as receptionists, nurses and doctors were not shown working software early, so the mismatch with real workflow was found only after delivery.
- **Value arises from use, not delivery.** Without a measurable effect on queues, the hospital got no return on investment, so the contract was not renewed - proving that 'delivered' is not the same as 'successful'.

(b) How delivered value should shape planning and prioritisation (4 marks)

- **Start with the outcome.** Define the goal ('reduce average waiting time by 25%') and use it as the test for every backlog item.
- **Prioritise by value-to-cost.** The Product Owner orders stories by their contribution to the goal, e.g., online appointment slots and a live queue display before low-impact reports.
- **Deliver the most valuable features first and release often** so value starts accruing early, and low-value features can be dropped.
- **Plan by outcome, not by output.** Review after each iteration: did waiting time improve? Adjust the next iteration accordingly.
- **Involve the customer continuously** (hospital staff in planning and demos) so priorities reflect reality; keep scope flexible, but the goal fixed.

(c) Two measures of organizational success (2 marks)

- **Average patient waiting time** (arrival to consultation) before vs after rollout, e.g., a drop from 45 to 30 minutes.
- **Staff/patient adoption and satisfaction** - percentage of appointments handled through the system, patient satisfaction score, or number of patients served per day (also contract renewal).

UNIT 1 - Q7. Case Study: EduSoft (10 marks)

Case: A 12-member team creates its own 'best-of-everything' method, taking one practice from several agile approaches, and drops pair work, testing discipline and short iterations because they 'feel slow'. After three months quality is falling and the practices do not support each other.

(a) What agile methods are; two examples (3 marks)

Agile methods are **lightweight, iterative approaches** that follow the Agile Manifesto: they deliver working software in short cycles, involve customers continuously, welcome change and rely on collaborative, self-organising teams. Each method is a **tested, interlocking bundle of practices**. Two well-known examples are **Scrum** (sprints, backlog, daily scrum, review and retrospective) and **Extreme Programming** (pair programming, TDD, continuous integration, small releases).

(b) Why designing your own method early is risky (4 marks)

- **Practices depend on each other.** In XP, TDD and continuous integration make short iterations safe; pairing spreads knowledge and acts as continuous review. EduSoft removed pairing, tests and short iterations, so nothing catches defects - hence falling quality.
- **'Feels slow' is a beginner's judgement.** At the Shu stage the team cannot yet judge which practices are essential. Practices that feel slow (testing, pairing) often save time later by preventing rework.
- **Cherry-picking leaves gaps.** Taking one practice from each method leaves no feedback loop: no short iterations means no frequent feedback, no tests means no confidence to change code.
- **No experience to guide adaptation.** A 12-member team also needs coordination which proven methods already provide; reinventing this is costly.
- **Evidence from EduSoft:** after 3 months quality is dropping and practices don't support each other - exactly the consequences predicted. (Shore's rule: *don't make your own method* until you have mastered an existing one.)

(c) Recommendation: how to adopt and adapt responsibly (3 marks)

- **Choose one established method** (e.g., Scrum with XP engineering practices) that fits the project and follow it **by the book for several iterations** (Shu).
- **Get a mentor/coach** to guide initial adoption.
- **Use retrospectives** to find real problems; change **one practice at a time**, observe the result, and keep a practice only if the evidence supports it.
- **Understand the purpose** of a practice before modifying it, and keep practices that reinforce each other (tests + CI + short iterations).
- **Reintroduce the dropped practices** (testing, pairing on risky code, 1-2 week iterations) to restore quality; consider splitting the 12 members into two smaller teams.

UNIT 1 - Q8. Case Study: CodeCraft (10 marks)

Case: Priya is new to agile and follows every rule strictly. Ravi has two years of experience and adjusts practices to fit the team. Anand, a veteran, creates new practices from scratch to suit the project.

(a) Stages of the road to mastery represented (4 marks)

Person	Behaviour	Stage	Explanation
Priya	Follows every rule strictly	**Shu** (Follow)	Beginner relying on explicit rules; focuses on how to practise correctly.
Ravi	Adjusts practices to fit the team	**Ha** (Break away)	Understands why practices exist, so can adapt them intelligently.
Anand	Creates new practices from scratch	**Ri** (Transcend)	Has internalised principles; invents practices to fit unique situations.

(b) Advice for each (4 marks)

- **Priya (Shu):** Keep practising the basics exactly; do not skip rules because they feel awkward. Work with a mentor or a pair partner, ask 'why' in retrospectives, and keep notes on what each practice achieves. Move towards Ha only after you can explain the purpose of each practice.
- **Ravi (Ha):** Change only one thing at a time, measure the effect, and share results in retrospectives. Study other methods (Scrum, Kanban, XP) to widen his options and guard against dropping practices just because they are hard. Mentor Priya - teaching deepens understanding.
- **Anand (Ri):** Keep experimenting, but document and test new practices with the team; seek peer review so personal bias is checked. Mentor and coach others, write about lessons learned, and stay curious - experts can still pick up bad habits and blind spots.

(c) Why a team should not skip the early stage (2 marks)

- Without Shu, a team lacks the **shared foundation and experience** to know which rules matter, so adaptations are guesses that often remove the very practices that protect quality (like EduSoft in Q7).
- The basic practices must be experienced to be understood; mastery of principles comes **through** practice, and skipping it leads to inconsistent, unstable ways of working.

UNIT 1 - Q9. Case Study: Finserve Technologies (10 marks)

Case: Finserve built a loan-processing portal, delivered on the planned date and within budget. Six months later loan officers still use spreadsheets because the portal doesn't match their workflow. Maintenance costs are high and the developers who built it are exhausted and have left.

(a) Kinds of success Finserve failed to achieve (4 marks)

Dimension	Evidence in the case	Result
Organizational success	Loan officers still use spreadsheets; portal does not match workflow	Little or no business value, wasted investment
Technical success	High maintenance cost	Poor design/quality; change is expensive (technical debt)
Personal success	Developers exhausted and left	Unsustainable pace; knowledge lost, team morale destroyed
Delivery success (achieved)	On date and within budget	Only this dimension was met - and it was not enough

This shows that meeting the deadline is only one part of success; the project 'succeeded' on paper but failed where it mattered.

(b) How an agile approach could have avoided each failure (6 marks)

Failure	Agile practices that would have helped	How it helps
Portal doesn't fit workflow (organizational)	Customer collaboration / on-site customer; user stories with acceptance tests; frequent demos; short iterations; backlog reprioritised by loan officers	Officers shape the system each iteration; mismatches are found in weeks, not after 6 months; value is delivered and used early.
High maintenance cost (technical)	TDD and automated tests; continuous integration; refactoring; simple design; collective code ownership; definition of done	Code stays clean and easy to change; defects found early; anyone can maintain it, lowering long-term cost.
Exhausted developers who left (personal)	Sustainable pace; realistic velocity-based planning; self-organizing team; retrospectives; pair programming; no hero overtime	Predictable workload, shared knowledge, healthier team; less turnover and loss of know-how.
Single focus on the date/budget	Value-driven prioritisation by the Product Owner; early partial releases; measure outcomes (adoption, processing time)	Success tracked by business outcomes rather than just schedule.

Conclusion: Agile reorients the project from 'deliver the plan on time' to 'deliver valuable, maintainable software with a healthy team', which is the broader definition of success.

UNIT 1 - Q10. Case Study: RetailNow (10 marks)

Case: RetailNow uses a long, fixed plan for its e-commerce app. Halfway through, a competitor launches a new payment feature and customers begin to expect it. The plan has no room for change, so the team is stuck.

(a) How agility helps a team respond to such change (5 marks)

- **Change is expected and cheap.** The product backlog is a living list. The new payment feature (e.g., UPI wallet / buy-now-pay-later) is added and prioritised immediately.
- **Short iterations give frequent decision points.** At the next sprint planning (1-2 weeks away), the team swaps lower-value items for the payment feature, instead of waiting for a long plan to finish.
- **Incremental, working releases.** The team can release the payment feature quickly and keep the rest of the app running, reducing the time that customers are without it.
- **Technical agility.** Automated tests, continuous integration and simple, refactored design let the team modify the payment module without breaking the checkout flow.
- **Customer and market feedback.** Regular reviews with business stakeholders and users show what customers value, so effort follows demand.
- **Self-organising team** can re-plan without lengthy approvals.

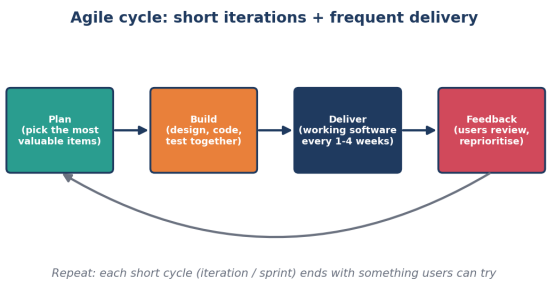


Fig 10.1 - Every iteration is an opportunity to re-prioritise

(b) Relevant agile values and principles (5 marks)

Values of the Agile Manifesto relevant here:

- **Responding to change over following a plan** - the central value: RetailNow's rigid plan is the problem.
- **Customer collaboration over contract negotiation** - work with business and customers on what matters now rather than hold to the original scope.
- **Working software over comprehensive documentation** - release usable features rather than wait for completed documentation.
- **Individuals and interactions over processes and tools** - empower the team to decide quickly.

Principles (from the 12 agile principles):

- **Welcome changing requirements, even late in development;** agile processes harness change for the customer's competitive advantage.
- **Deliver working software frequently** (every few weeks rather than months).
- **Satisfy the customer through early and continuous delivery** of valuable software.
- **Business people and developers work together daily** to decide priorities.
- **Continuous attention to technical excellence and good design** enhances agility; **simplicity** - maximising work not done - keeps the product flexible.
- **Regular reflection** (retrospectives) to adjust behaviour and process.

Conclusion: RetailNow should abandon the fixed long-range plan for a backlog-driven, iterative approach in which change is handled routinely.

UNIT 2 - Q1. The Agile Manifesto and its Principles (10 marks)

Question: Discuss the Agile Manifesto and its principles.

Background

In February 2001, seventeen software practitioners (including Kent Beck, Martin Fowler, Ken Schwaber, Jeff Sutherland, Alistair Cockburn and Ward Cunningham) met at Snowbird, Utah and agreed on a short statement of values for better ways of developing software, called the **Manifesto for Agile Software Development**. They represented XP, Scrum, DSDM, Crystal, FDD and others, and found common ground in lightweight, people-centred approaches.

The four values

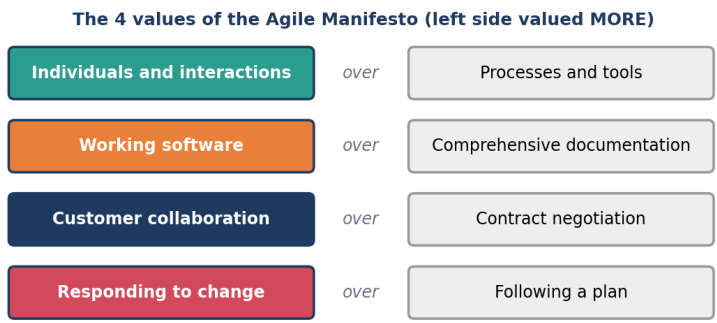


Fig U2.1 - Values of the Agile Manifesto

The manifesto states that while there is value in the items on the right, the items on the left are valued **more**:

Value	Meaning	Example
Individuals and interactions over processes and tools	Skilled, communicating people matter more than rigid procedures or tools.	A quick conversation resolves a doubt faster than a ticket workflow.
Working software over comprehensive documentation	Progress is measured by software that runs; keep just enough documentation.	Demo of a working login beats a 50-page design document.
Customer collaboration over contract negotiation	Work with the customer continuously rather than rely on a fixed contract.	Weekly review with the client to refine requirements.
Responding to change over following a plan	Plans are guides; change is welcomed.	Re-prioritising the backlog when the market changes.

The twelve principles

#	Principle (summary)	Meaning in practice
1	Satisfy the customer through early and continuous delivery of valuable software	Release useful increments early and often.
2	Welcome changing requirements, even late	Change is a competitive advantage, not a threat.
3	Deliver working software frequently (weeks rather than months)	Short iterations, fast feedback.
4	Business people and developers work together daily	Product Owner is part of the team's day.
5	Build projects around motivated individuals; give them support and trust	Empower, don't micro-manage.
6	Face-to-face conversation is the best way to convey information	Co-location, stand-ups, pairing; video if remote.
7	Working software is the primary measure of progress	Demos, not status reports.

#	Principle (summary)	Meaning in practice
8	Agile processes promote sustainable development; a constant pace	No death marches; avoid burnout.
9	Continuous attention to technical excellence and good design	TDD, refactoring, code reviews.
10	Simplicity - the art of maximising the amount of work not done	Build only what is needed (YAGNI).
11	The best architectures, requirements and designs emerge from self-organising teams	Team decides how to do the work.
12	At regular intervals the team reflects and adjusts	Retrospectives and continuous improvement.

Significance

The Manifesto is not a method; it is a **mindset and a set of values** that guide choices in methods such as Scrum, XP and Kanban. It shifted the industry from heavyweight, plan-driven processes to collaborative, adaptive and value-focused development.

UNIT 2 - Q2. Simple Design and User Stories (10 marks)

Question: Explain Simple Design and User Stories in agile development.

A. Simple Design

Simple design (an XP practice) means building the **simplest design that satisfies the current requirements**, and improving it continuously, instead of designing a large, complex architecture up front for imagined future needs. It follows the principle '*maximise the work not done*' and YAGNI (You Aren't Gonna Need It).

Kent Beck's rules of simple design

- **Passes all the tests** - the code works as intended.
- **Reveals intention** - clear names and structure; easy to read.
- **No duplication** (DRY) - each idea expressed once.
- **Fewest elements** - no unnecessary classes, methods or features.

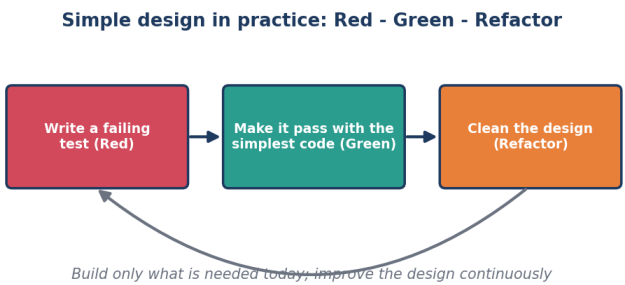


Fig U2.2 - Simple design is sustained through TDD and refactoring

Benefits

- Easier to understand, test and change - supports changing requirements.
- Less wasted effort on features that are never used.
- Lower defects and maintenance cost; faster delivery.

Example: For the attendance system, store attendance in one table with (student, date, subject, status) and a simple service to compute percentage. Only if reports later need more should a separate reporting design be added.

B. User Stories

A **user story** is a short, simple description of a feature told from the perspective of the user. It captures *who* wants *what* and *why*, and acts as a promise to hold a conversation about the details.

Template: *As a <role>, I want <feature>, so that <benefit>.*

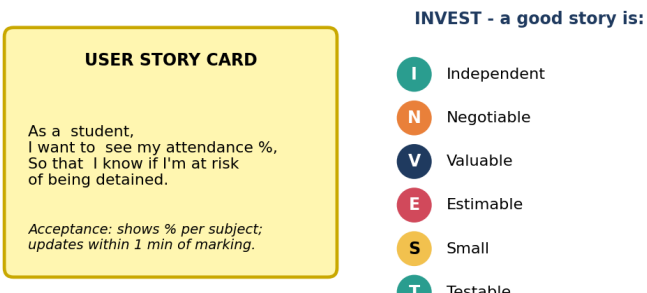


Fig U2.3 - A user story card and the INVEST criteria

The 3 Cs of a story

- **Card** - the written one-line story on an index card/ticket.
- **Conversation** - discussion between team and customer to elicit details.
- **Confirmation** - acceptance criteria / tests to verify it is done.

Characteristics, estimation and use

- Good stories follow **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable).
- Stories are placed in the product backlog, **estimated** in story points, prioritised by value and fitted into sprints. Large stories (epics) are split into smaller ones.
- **Example acceptance criteria:** 'Given a student is logged in, when they open the attendance page, then the percentage per subject is displayed.'

Why stories rather than long requirement documents

They encourage conversation, are easy to reprioritise, keep the focus on user value, and fit short iterations - supporting 'working software over documentation' and 'customer collaboration'.

UNIT 2 - Q3. Types of Agile Testing (10 marks)

Question: Discuss the different types of agile testing process.

Introduction

In agile, testing is **not a separate phase at the end**; it is a continuous activity performed by the whole team in every iteration. Testers, developers and the customer collaborate from the start, and automation provides fast feedback. The popular classification is the **Agile Testing Quadrants** (Brian Marick, popularised by Lisa Crispin and Janet Gregory).

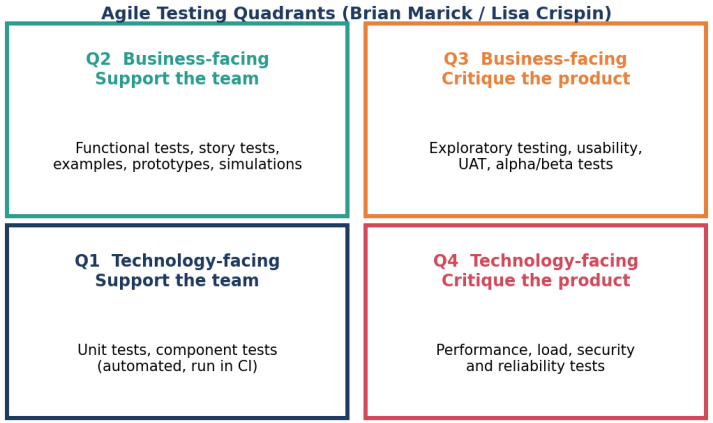


Fig U2.4 - Agile testing quadrants

Types of agile testing

Quadrant / type	Purpose	Typical examples and tools
Q1 Unit and component tests (technology-facing, support the team)	Verify small pieces of code work correctly; written by developers, often test-first (TDD). Run on every build.	JUnit, NUnit, PyTest
Q2 Functional / acceptance tests (business-facing, support the team)	Check that stories meet acceptance criteria; use examples from customers. Includes **ATDD** and **BDD**.	Cucumber, FitNesse, SpecFlow
Q3 Exploratory, usability and user-acceptance tests (business-facing, critique the product)	Human, creative testing to find what automation misses; assess usability and real-world fit; alpha/beta testing.	Session-based exploratory testing, UAT demos
Q4 Non-functional tests (technology-facing, critique the product)	Performance, load, stress, security, scalability and reliability.	JMeter, Gatling, OWASP ZAP

Other important agile testing practices

- **Test-Driven Development (TDD):** write failing test -> code -> refactor.
- **Acceptance Test-Driven Development (ATDD) / Behaviour-Driven Development (BDD):** tests are written collaboratively in 'Given-When-Then' form before development.
- **Continuous Integration testing:** automated tests run on each commit to catch integration defects early.
- **Regression testing:** re-running automated tests every iteration to ensure that new changes haven't broken existing features.
- **Exploratory testing** and **pair testing** between tester and developer.

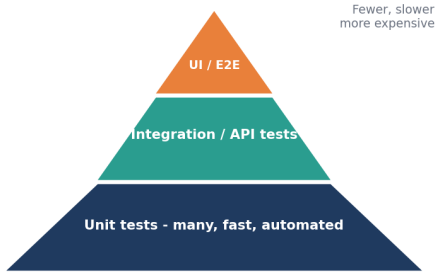


Fig U2.5 - Test automation pyramid: many unit tests, fewer UI tests

Key principles

- Whole-team responsibility for quality; testing from day one; early and frequent feedback; automate repeatable tests; keep testing in the definition of done.

UNIT 2 - Q4. Types of Agile Tools (10 marks)

Question: Discuss the different types of agile tools.

Introduction

Agile favours people and interactions over tools, yet the right tools help teams plan, collaborate, integrate, test and deliver faster, especially when teams are distributed. Agile tools can be grouped by purpose.

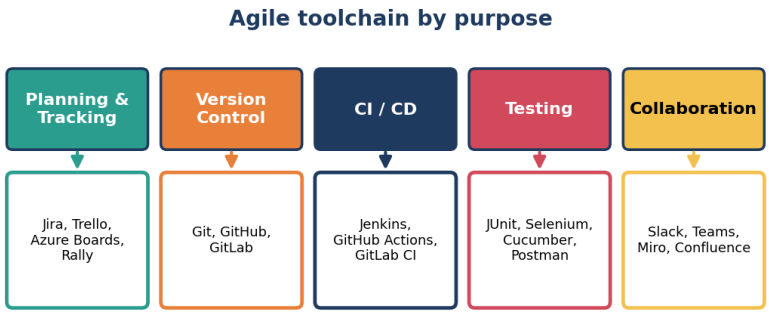


Fig U2.6 - Types of agile tools

Type	Purpose	Examples and typical use
Project planning and tracking tools	Manage product backlog, sprints, task boards, burn-down charts, velocity and reports.	Jira, Azure DevOps Boards, Trello, Rally, Asana. Create user stories, plan sprints, track progress.
Version control tools	Store source code, track changes, support branching and merging, enable collaboration.	Git, GitHub, GitLab, Bitbucket.
Continuous integration / delivery (CI/CD) tools	Automatically build, test and deploy every change to give rapid feedback.	Jenkins, GitHub Actions, GitLab CI, CircleCI.
Test automation tools	Automate unit, API, UI and performance tests; support TDD/BDD.	JUnit, Selenium, Cucumber, Postman, JMeter.
Collaboration and communication tools	Instant communication, video meetings, shared documentation and virtual whiteboards for distributed teams.	Slack, Microsoft Teams, Zoom, Confluence, Miro.
Code quality and review tools	Static analysis, code coverage, code review to maintain technical excellence.	SonarQube, ESLint, Pull Requests.
Retrospective and estimation tools	Support retrospectives, planning poker and team health checks.	Retrium, Miro boards, PlanningPoker.online.

Criteria for selecting tools

- Fits the team's way of working (not the reverse); simple and transparent.
- Integrates with other tools; supports distributed teams.
- Cost, security, learning curve and community support.

Note: A tool cannot make a team agile; a physical board with sticky notes can be as effective for a co-located team.

UNIT 2 - Q5. Genesis of Agile and Limitations of Waterfall (10 marks)

Question: Trace the genesis of Agile. Explain the limitations of the waterfall model that led to lightweight methods.

Genesis of agile

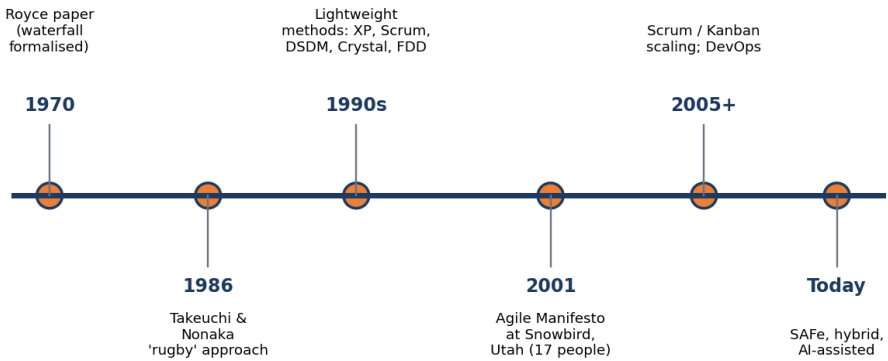


Fig U2.7 - Timeline: from waterfall to agile

- **1970:** Winston Royce's paper described a sequential model (later called waterfall) - ironically, he warned it was risky for large projects. Industry adopted it widely and heavily (e.g., defence standards).
- **1980s-90s:** Growing evidence of failed, late and over-budget projects. Iterative and incremental ideas appeared: spiral model, RAD, and Takeuchi and Nonaka's 1986 'rugby' approach (the inspiration for Scrum).
- **Early-mid 1990s:** Lightweight methods emerged: Scrum (Schwaber and Sutherland), DSDM, Crystal (Cockburn), Feature-Driven Development, and Extreme Programming (Kent Beck, 1996 Chrysler C3 project).
- **2001:** 17 practitioners met at Snowbird, Utah and issued the Agile Manifesto, uniting the lightweight methods under one name 'Agile'.
- **After 2001:** Wide adoption: Scrum became dominant; Lean and Kanban entered software; DevOps and continuous delivery extended agility to operations; scaling frameworks (SAFe, LeSS) evolved.

Limitations of the waterfall model

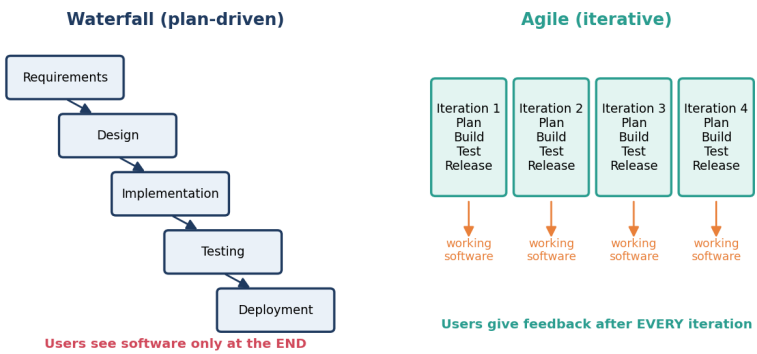


Fig U2.8 - Waterfall vs agile

Limitation	Explanation	Consequence
Rigid, sequential phases	Each phase must finish before the next begins; going back is costly.	Cannot adapt to change.

Limitation	Explanation	Consequence
Requirements frozen up front	Customers rarely know all needs at the start.	Product built to outdated or wrong requirements.
Late feedback and delayed testing	Working software appears only near the end; testing is last.	Defects and misunderstandings found when they are most expensive to fix.
High risk	Big-bang delivery; risk is deferred until integration/testing.	Late failures, schedule overruns.
Heavy documentation	Extensive documents at each phase.	Slow; documents become outdated; effort not spent on value.
Limited customer involvement	Customer participates at the start and at the end.	Final product may not meet real needs.
Poor fit with changing business and technology	Long cycles (months-years).	Product obsolete on release.
Weak team collaboration	Hand-offs between analysts, designers, coders, testers.	Misunderstandings, blame, low morale.

Conclusion

These limitations motivated lightweight (agile) methods, which use **iterative and incremental delivery, continuous customer collaboration, early testing and adaptive planning**. Waterfall is still suitable for projects with stable, well-understood requirements (e.g., some regulated or hardware-bound projects).

UNIT 2 - Q6. Case Study: Online Attendance Management System

Case: A college needs a system where teachers mark attendance and students check their attendance %. Only basic requirements are given initially; teachers later suggest attendance reports, low-attendance notifications and timetable integration. The team uses Agile.

1. Agile Manifesto - how the four values apply

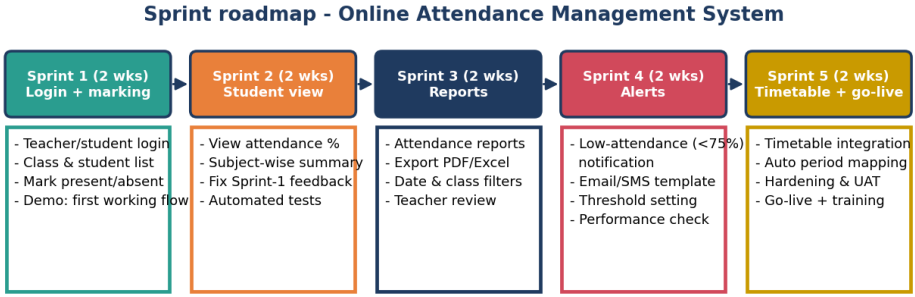
Value	Application in this project
Individuals and interactions over processes and tools	The team sits with teachers and students regularly; decisions are made through quick discussions and demos rather than long approval processes.
Working software over comprehensive documentation	After each sprint a usable part is shown (mark attendance, view %). Documentation is limited to user stories, acceptance criteria and a short user guide.
Customer collaboration over contract negotiation	Teachers are the Product Owner/reviewers. Their suggestions (reports, alerts, timetable) are welcomed as backlog items instead of being treated as a scope dispute.
Responding to change over following a plan	The plan is a prioritised backlog. New features are slotted into later sprints; the release plan is revised after every review.

2. The 12 agile principles applied

#	Principle	Application in this project
1	Customer satisfaction through early, continuous delivery	Working basic version in Sprint 1-2 lets teachers and students start using and giving feedback early.
2	Welcome changing requirements	Reports, notifications and timetable integration are added late without penalty; backlog is reordered.
3	Deliver working software frequently	Delivery every 2 weeks: mark attendance, view %, reports, alerts, timetable.
4	Business people and developers work together daily	A teacher representative attends planning and review and answers questions daily.
5	Motivated individuals, trusted and supported	Motivated teachers and developers are trusted to decide the how; a Scrum Master removes blockers.
6	Face-to-face conversation	Daily stand-up, short demos to teachers, pairing on tricky integration instead of long emails.
7	Working software is the measure of progress	Progress is shown by working features (e.g., 'teachers can mark attendance today'), not percent-complete reports.
8	Sustainable pace	Steady sprint scope based on velocity; no last-minute overtime before demos.
9	Technical excellence and good design	Automated tests for percentage calculation, CI builds, refactoring as features are added.
10	Simplicity - maximise work not done	Basic features first; no complex analytics until requested; reuse the same attendance table for reports.
11	Self-organising teams	The team decides task split (UI, DB, notifications) and self-assigns stories.
12	Regular reflection and adjustment	Retrospective after each sprint, e.g., improve how teachers give feedback.

3. Sprint plan

Assumptions: Scrum, 2-week sprints, team of 5-6 (Product Owner from the department, Scrum Master, developers/tester). Each sprint has Planning, Daily Scrum, Review and Retrospective; the Definition of Done includes tested, reviewed and demoed. Priority is by value; later sprints stay flexible so new suggestions can be absorbed.



Sprint-wise roadmap (backlog is re-prioritised after every sprint review)

Sprint	Sprint goal	Key user stories	Outcome (increment)
1	Login and mark attendance	Teacher logs in; selects class; marks present/absent. Student login.	Teachers can record attendance.
2	Student view	Student views overall and subject-wise attendance %.	Students can check attendance.
3	Reports	Teacher/admin generates attendance reports (date, class); export.	Reports available.
4	Alerts	Low-attendance (e.g., <75%) notification by email/SMS; configurable threshold.	Automated warnings.
5	Timetable and release	Timetable integration to auto-map periods; UAT, training, deployment.	Release 1.0 live in college.

UNIT 2 - Q7. Case Study: Online Learning Platform

Case: A software company develops an online learning platform for a college, with lectures, study materials, quizzes and marks. Only basic requirements are given first; teachers and students later suggest live classes, discussion forums and notifications. The team follows the Agile Manifesto and principles.

1. Agile Manifesto - how the four values apply

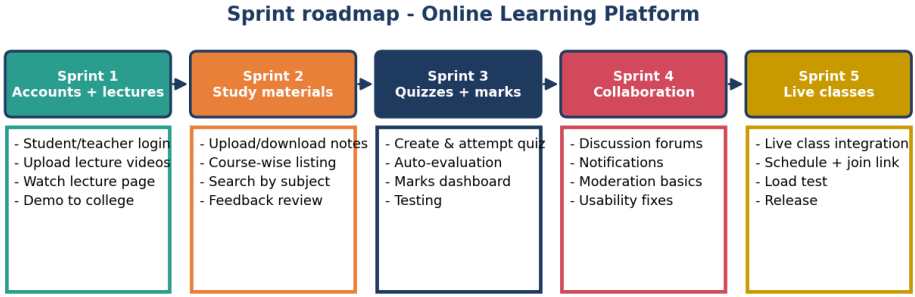
Value	Application in this project
Individuals and interactions over processes and tools	Developers, teachers and students interact frequently through short feedback sessions; the team uses simple tools (board, chat).
Working software over comprehensive documentation	Each sprint ends with a usable feature (e.g., watch a lecture, attempt a quiz) rather than a thick requirement document.
Customer collaboration over contract negotiation	Teachers and students are involved throughout: they review sprint demos and contribute suggestions.
Responding to change over following a plan	Live classes, forums and notifications are welcomed late; the roadmap is revised without a contract dispute.

2. The 12 agile principles applied

#	Principle	Application in this project
1	Customer satisfaction through early, continuous delivery	Early release of lectures and materials gives students value in the first weeks.
2	Welcome changing requirements	Suggestions for live classes, forums and notifications are accepted even after development has begun.
3	Deliver working software frequently	Every 2 weeks a new working capability goes to a pilot class.
4	Business people and developers work together daily	A teacher acts as Product Owner and joins planning/review; student representatives give feedback.
5	Motivated individuals, trusted and supported	A motivated team is trusted to choose video-streaming and quiz technology.
6	Face-to-face conversation	Direct conversations (or video) with teachers rather than lengthy emails.
7	Working software is the measure of progress	Progress is measured by working features students can use, e.g., quizzes taken.
8	Sustainable pace	Fixed sprint length and realistic commitment prevent overload.
9	Technical excellence and good design	Clean modular design (content, quiz, marks modules); automated tests and CI.
10	Simplicity - maximise work not done	Start with simple video hosting and file download; add complexity only when needed.
11	Self-organising teams	The team organises itself - front-end, back-end, QA - and shares responsibility.
12	Regular reflection and adjustment	Retrospectives address issues like slow video loading and improve process.

3. Sprint plan

Assumptions: Scrum, 2-week sprints, team of 5-6 (Product Owner from the department, Scrum Master, developers/tester). Each sprint has Planning, Daily Scrum, Review and Retrospective; the Definition of Done includes tested, reviewed and demoed. Priority is by value; later sprints stay flexible so new suggestions can be absorbed.



Sprint-wise roadmap (backlog is re-prioritised after every sprint review)

Sprint	Sprint goal	Key user stories	Outcome (increment)
1	Accounts and lectures	Login for students/teachers; teachers upload lectures; students watch.	Lectures usable.
2	Study materials	Upload and download notes; course-wise listing; search.	Materials available.
3	Quizzes and marks	Create and attempt quizzes, auto-evaluate, marks dashboard.	Assessment working.
4	Collaboration	Discussion forum, notifications for new content/quizzes.	Engagement features.
5	Live classes and release	Live class integration, scheduling, load testing, release.	Release 1.0 with live classes.

UNIT 2 - Q8. Case Study: E-Cart Application

Case: A company develops an E-Cart app (register, browse, cart, payments, order tracking). The client gives basic requirements first, then the business team and customers suggest wishlist, reviews, discount coupons, multiple payment options and order tracking. The team follows the Agile Manifesto and principles.

1. Agile Manifesto - how the four values apply

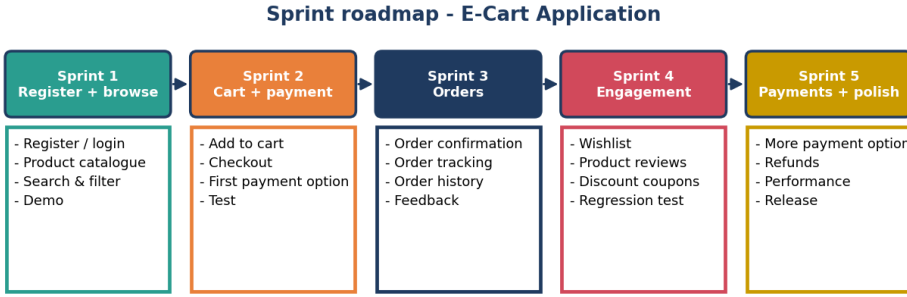
Value	Application in this project
Individuals and interactions over processes and tools	Developers, business team and customer representatives collaborate frequently; decisions are made on the basis of conversation and user behaviour.
Working software over comprehensive documentation	A working catalogue and checkout appear early; documentation is lightweight (stories, API notes).
Customer collaboration over contract negotiation	The client/product owner works with the team on priorities; feedback from real customers is gathered and used.
Responding to change over following a plan	Wishlist, coupons and reviews are added when business wants them; the backlog changes with market and festival-season needs.

2. The 12 agile principles applied

#	Principle	Application in this project
1	Customer satisfaction through early, continuous delivery	Core shopping flow (browse -> cart -> pay) is released first so the business can start selling.
2	Welcome changing requirements	New features (wishlist, coupons, more payment options) are welcome at any time as backlog items.
3	Deliver working software frequently	Frequent releases every 2 weeks, possibly behind feature flags.
4	Business people and developers work together daily	Business team (sales, marketing) takes part in planning and reviews; a Product Owner decides priority.
5	Motivated individuals, trusted and supported	Cross-functional team is given trust and tools to decide technical solutions for payment and cart.
6	Face-to-face conversation	Daily stand-ups, quick calls with the payment gateway/business teams.
7	Working software is the measure of progress	Working checkout and orders placed in test are the progress measure.
8	Sustainable pace	Planned load per sprint; avoid heavy overtime before sale events.
9	Technical excellence and good design	Strong automated testing on cart and payments; secure coding; CI/CD.
10	Simplicity - maximise work not done	Implement one payment method first; add others later; avoid unused features.
11	Self-organising teams	The team self-organises around stories (cart, orders, payments).
12	Regular reflection and adjustment	Retrospectives analyse issues like abandoned carts and adjust the process.

3. Sprint plan

Assumptions: Scrum, 2-week sprints, team of 5-6 (Product Owner from the department, Scrum Master, developers/tester). Each sprint has Planning, Daily Scrum, Review and Retrospective; the Definition of Done includes tested, reviewed and demoed. Priority is by value; later sprints stay flexible so new suggestions can be absorbed.



Sprint-wise roadmap (backlog is re-prioritised after every sprint review)

Sprint	Sprint goal	Key user stories	Outcome (increment)
1	Register and browse	Registration/login, product catalogue, search and filter.	Customers can browse.
2	Cart and payment	Add to cart, checkout, first payment method.	End-to-end purchase.
3	Orders	Order confirmation, history and tracking.	Order tracking live.
4	Engagement	Wishlist, product reviews, discount coupons.	Business engagement features.
5	Payments and polish	Additional payment options, refunds, performance and security tests.	Release with multiple payments.

UNIT 2 - Q9. Case Study: Doctor Appointment Booking App

Case: A hospital wants a mobile app for booking doctor appointments. Initial requirements: patient registration and login, doctor search, doctor profile, appointment booking and appointment history. The team follows the Agile Manifesto and principles.

1. Agile Manifesto - how the four values apply

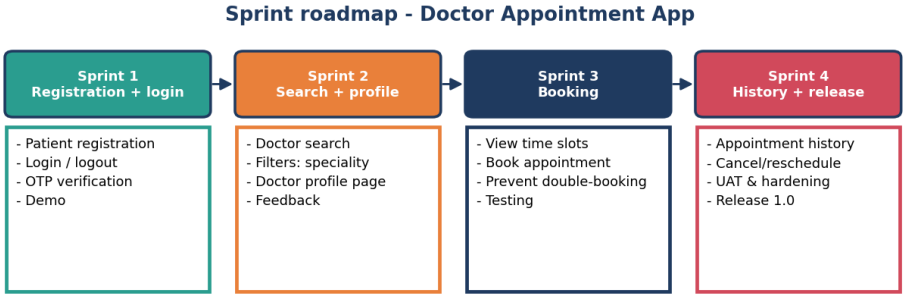
Value	Application in this project
Individuals and interactions over processes and tools	Developers, doctors, front-desk staff and patients are consulted directly; the team prioritises usability for patients of all ages.
Working software over comprehensive documentation	A clickable, working app is shown every sprint; documentation is limited to key flows, API and privacy notes.
Customer collaboration over contract negotiation	Hospital administration and doctors continuously review; details of slot rules are worked out through conversations.
Responding to change over following a plan	Later needs (cancellations, reminders, teleconsultation) can be added without renegotiating a contract.

2. The 12 agile principles applied

#	Principle	Application in this project
1	Customer satisfaction through early, continuous delivery	Early release of registration, search and booking provides value quickly (fewer phone calls).
2	Welcome changing requirements	Changes such as slot rules or cancellation policy are accepted at any time.
3	Deliver working software frequently	Release a working increment every 2 weeks to a pilot department.
4	Business people and developers work together daily	Hospital representative (Product Owner) works daily with developers; doctors review profiles/slots.
5	Motivated individuals, trusted and supported	Motivated team is given a clear goal (reduce phone bookings) and freedom to solve it.
6	Face-to-face conversation	Direct conversation with doctors and front-desk staff instead of long documents.
7	Working software is the measure of progress	Real, bookable appointments in the test system measure progress.
8	Sustainable pace	Sustainable pace across 4 sprints; no rushed delivery.
9	Technical excellence and good design	Strong testing for double-booking, privacy and security of medical data; clean design.
10	Simplicity - maximise work not done	Only the five requested features first; no extras until validated.
11	Self-organising teams	Team organises itself around stories (auth, search, booking) and shares code ownership.
12	Regular reflection and adjustment	Retrospectives after pilot feedback (e.g., confusing slot display) and adaptation.

3. Sprint plan

Assumptions: Scrum, 2-week sprints, team of 5-6 (Product Owner from the department, Scrum Master, developers/tester). Each sprint has Planning, Daily Scrum, Review and Retrospective; the Definition of Done (with data privacy checks) includes tested, reviewed and demoed. Priority is by value; later sprints stay flexible so new suggestions can be absorbed.



Sprint-wise roadmap (backlog is re-prioritised after every sprint review)

Sprint	Sprint goal	Key user stories	Outcome (increment)
1	Registration and login	Patient registration, login/logout, OTP verification.	Patients can sign in.
2	Search and profile	Search doctors by name/speciality; view doctor profile.	Doctors discoverable.
3	Booking	View available slots, book appointment, prevent double-booking, confirmation.	Appointments bookable.
4	History and release	Appointment history, cancel/reschedule, UAT and hardening; release.	Release 1.0 in hospital.

Global Academy of Technology

(An Autonomous Institution, affiliated to VTU, Belagavi)

Department of Computer Science & Engineering

Question Bank: Answer Key

MODULE 2: Extreme Programming (XP)

Branch	Semester	Course Name	Course Code
CSE	V	Agile Technology	BCS24505D

How to use this answer key. Each answer follows a 10-mark exam pattern: a short introduction, the main points under clear headings, a relatable example (mostly Indian everyday or campus scenarios), and a diagram wherever it helps. For the scenario questions (Part 2) the answers are labelled (a) to (d) exactly as in the question. In an exam you can reproduce the headings, the main points and a simplified version of the diagram.

Scope note. The uploaded question bank contains questions only for Module 2 (20 questions). The question tables for Modules 3, 4 and 5 were blank in the file, so they are not covered here. Send those questions and I can add them.

Contents

Part 1: Theory Questions . . . 3

Q1. XP life cycle . . . 3

Q2. Types of coaches . . . 4

Q3. Key concepts of XP . . . 5

Q4. Theory of Constraints and mindfulness . . . 6

Q5. XP roles and responsibilities . . . 7

Q6. Six prerequisites for XP . . . 8

Q7. Recommendations for adopting XP . . . 9

Q8. Team size and full-time participation . . . 10

Q9. Applying XP practices in an ongoing project . . . 11

Q10. Challenges of adopting XP . . . 12

Part 2: Scenario-Based Questions . . . 13

Q1. Mobile banking: technical debt and refactoring . . . 13

Q2. Food delivery: timeboxing and prioritisation . . . 14

Q3. Partial XP adoption: pairing, testers, refactoring . . . 16

Q4. Hospital system: distributed team, absent customer . . . 17
Q5. Velocity and bottleneck . . . 19
Q6. Missing management support . . . 20
Q7. Team of 15, no coach, solo modules . . . 21
Q8. Legacy student information system . . . 22
Q9. E-commerce platform: readiness and adoption plan . . . 24
Q10. Government project with mandatory phases . . . 25

Quick Revision Sheet . . . 27

PART 1: Theory Questions

Q1 [L2 | 10 marks]

Explain the Extreme Programming (XP) life cycle.

Extreme Programming (XP) is an agile method for small to medium teams. Unlike the waterfall model (requirements, then design, then coding, then testing), XP does **not** run in separate phases. The team repeats a short cycle of *plan, build, demo, improve* and releases working software regularly. The XP life cycle has five stages.

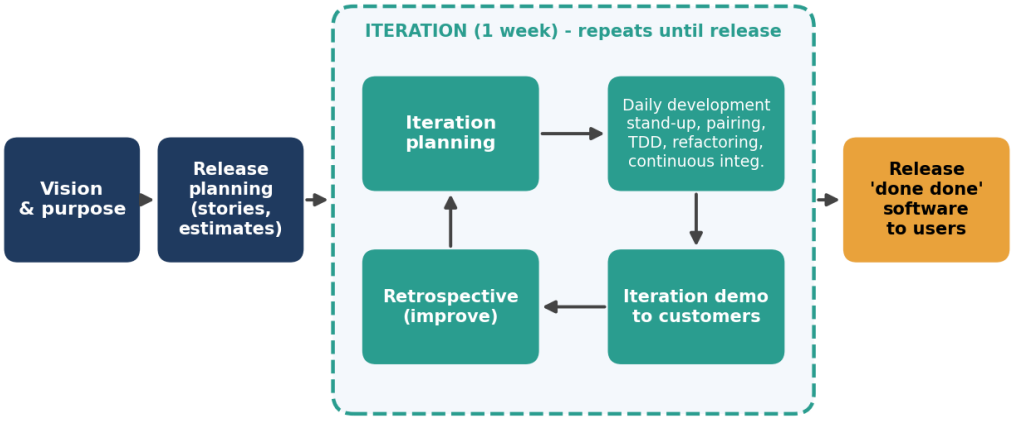


Fig. 1: XP life cycle (vision, release planning, repeating iterations, release)

1. Vision

The product manager explains *why* the product exists, who will use it and what success looks like. **Example:** for a college-fest app "Fest360" the vision is "one app where students register, pay and get live updates for every event".

2. Release planning

Customers write **user stories** (small features), programmers estimate them in story points, and the customer prioritises them. Using the team's **velocity** the team forecasts how much can be released by a given date (typically 2-3 months or less). **Example:** Fest360 stories: register for an event, pay online, download a QR ticket, see live schedule.

3. Iterations (usually 1 week)

- **Iteration planning:** the team picks the stories it can finish, based on past velocity ("yesterday's weather").
- **Daily development:** daily stand-up, pair programming, test-driven development (TDD), refactoring, continuous integration (CI) and collective code ownership. The on-site customer answers questions immediately.
- **Iteration demo:** the team shows *working* software to customers and stakeholders and collects feedback.
- **Retrospective:** the team discusses what went well, what did not and chooses one or two improvements for the next iteration.

4. Release

Software that is **"done done"** (coded, tested, integrated, accepted by the customer) is released to real users, ideally every few weeks or even every iteration. **Example:** Fest360 version 1 with registration + payment is released first; ticket QR codes follow in the next release.

5. Feedback and the next cycle

User feedback, changed priorities and new ideas go back into release planning. The cycle repeats until the product is retired.

Activity	How often	Who	Output
Vision	Once (revisited when needed)	Product manager, stakeholders	Shared purpose
Release planning	Every release (weeks to a few months)	Whole team	Prioritised stories + release forecast
Iteration planning	Start of each iteration	Whole team	Committed stories
Stand-up / pairing / TDD / CI	Daily	Programmers, testers	Working, tested code
Demo + retrospective	End of each iteration	Team + customers	Feedback + improvements

Q2 [L2 | 10 marks]

Identify the types of Coaches in Extreme Programming (XP) with suitable real-time examples.

A coach helps the team **learn and consistently apply** XP. Think of a cricket team: the *batting coach* improves technique, while the *team manager* handles logistics so players can focus on the game. XP has the same split. The two coaching roles are the **programmer-coach** and the **project manager** (plus an optional external guide during adoption).

	Programmer-coach (technical coach)	Project manager (organisational coach)
Focus	Technical practices and engineering discipline	Organisation, communication, resources
Typical person	Experienced programmer who is also a good teacher	Person with people and organisational skills; understands XP
Responsibilities	Teach pairing, TDD, refactoring, simple design; spot bad habits; help set up CI and the build; mentor juniors; lead retrospectives on technical issues	Remove obstacles; shield the team from interruptions; arrange team room, equipment, budget; liaise with management and external parties; report progress (velocity, release forecast); support customers and team relationships
Does not do	Does not give orders or take over the keyboard; guides so the team learns	Does **not** make technical decisions or assign tasks; the team self-organises

Real-time examples

- **Programmer-coach:** In a team building a food-ordering app, a senior developer notices the new joiners skip writing tests when the deadline is near. She pairs with them for a day, shows how to write the test first for the "apply coupon" story, and raises "tests are being skipped" in the retrospective. Within two iterations the team's defect count drops.
- **Project manager:** The same team needs a shared room, a CI server and two extra monitors for pairing. The project manager gets these approved, tells other departments not to pull developers into side meetings, and shows management the velocity chart each month instead of asking for long status reports.
- **External guide (adoption phase):** A company new to XP hires an agile mentor for the first three months. The mentor coaches the team and the programmer-coach, and then steps back once the in-house coach is confident.

Key point: coaches *enable* the team; they do not control it. Their success is measured by how soon the team no longer needs them.

Q3 [L2 | 10 marks]

Explain the key concepts of Extreme Programming (XP) with suitable real-time examples.

Seven key concepts underpin how an XP team plans work and keeps its code healthy.

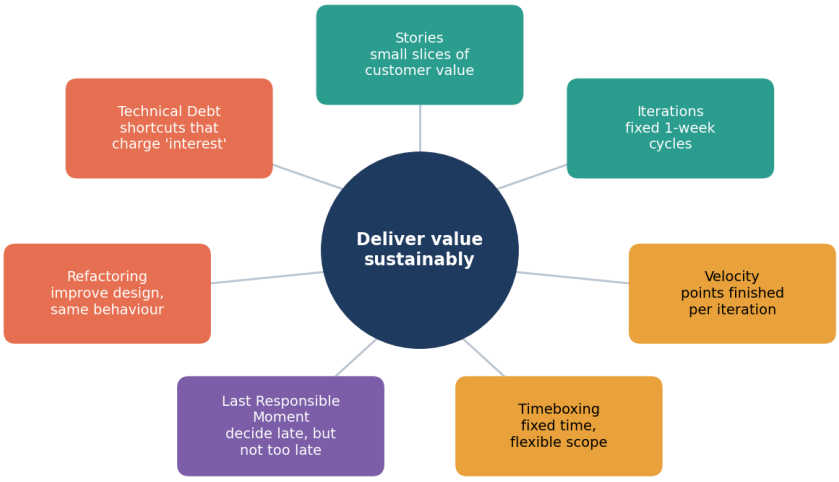


Fig. 2: Seven key concepts of XP

Concept	Meaning	Real-time example
Stories	A short description of something valuable to the customer, written in their language, with acceptance criteria. Small enough to finish in a few days.	"As a student, I want to download my hall ticket so that I can enter the exam hall." Acceptance: PDF has photo, USN, subject and seat number.
Iterations	Fixed-length cycles (normally 1 week) in which the team plans, builds, tests and demos a small set of stories.	Like a weekly mess menu: same rhythm every week, small and repeatable, and you can improve next week.
Velocity	The number of story points the team *completes* (done done) per iteration. Used to forecast; not a performance score.	A cab app shows ETA using recent traffic. Similarly, if a team finished 18, 20 and 19 points, it plans about 19 next time.
Timeboxing	Fix the time and cost, and flex the scope. When time runs out, stop and ship what is done.	A 3-hour exam: you cannot extend the time, so you answer the most valuable questions first.
Last Responsible Moment (LRM)	Delay a decision until the latest point at which delaying further would cause harm, so it is made with the most information.	Do not finalise the wedding menu months ahead; decide when the guest count is known, but early enough for the caterer to prepare.
Refactoring	Improving the internal structure of code without changing its behaviour, in small safe steps protected by tests.	Splitting a 200-line `calculateTotal()` method into small named methods, or cleaning your desk without changing what you work on.
Technical debt	The future cost of shortcuts in design/code. Like a loan: you get speed now but pay "interest" later as slower changes and more bugs.	Hard-coding GST in ten places to meet a demo. Next year the rate changes and ten places must be fixed.

Q4 [L2 | 10 marks]

Discuss the Theory of Constraints and mindfulness in Extreme Programming (XP) with suitable real-time examples.

A. Theory of Constraints (TOC)

Proposed by Eliyahu Goldratt, TOC says that every process has **one constraint (bottleneck)** that limits the output of the whole system. Improving any other step does not increase total output; only improving the bottleneck does.

Everyday example: a restaurant kitchen has four cooks but only one tandoor. No matter how fast the cooks chop and prepare, the number of rotis served per hour is decided by the tandoor.

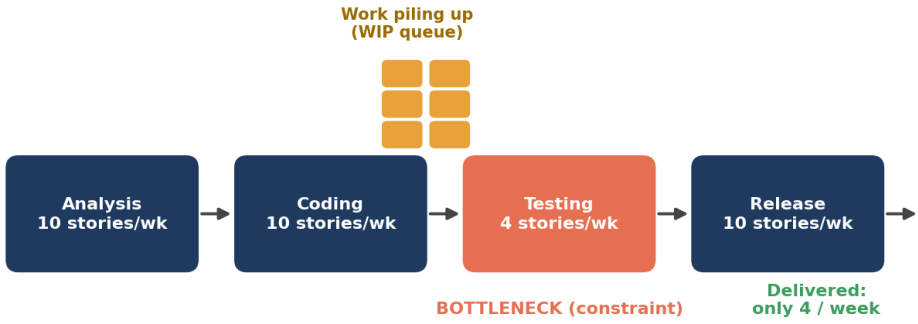


Fig. 3: Testing is the constraint; faster coding only builds a queue of untested work

In software: developers write 10 stories a week, but testers can verify only 4. The team delivers 4 stories a week; the remaining work waits as *work in progress (WIP)* and its defects are found late.

Five focusing steps	Meaning	Software example
1. Identify	Find the bottleneck	Cycle-time data shows stories wait 5 days for testing
2. Exploit	Get the most out of the bottleneck	Testers stop doing manual repetitive checks; remove interruptions
3. Subordinate	Align everything else to the bottleneck	Developers stop starting new stories when testing is full; they help test or fix bugs
4. Elevate	Increase the bottleneck's capacity	Automate regression tests, add a tester, pair testers with programmers
5. Repeat	A new constraint will appear; find it	Next bottleneck may be customer feedback or deployment

How XP applies TOC: whole-team approach (testers and customers sit with programmers), *done done* stories, small stories, TDD (testing moves earlier), continuous integration, and limiting WIP so the whole system flows evenly.

B. Mindfulness

Mindfulness in XP means **paying deliberate attention to what is happening right now**: how we work, how the code looks and how the team feels, and then acting on what we notice. It is the habit that makes improvement possible.

- **Pair programming:** while one person types, the partner watches for bugs and design smells.
- **Stand-ups and informative workspace:** visible task boards and build-status screens make problems obvious.
- **Retrospectives:** a regular pause to ask "what is working and what is not?".
- **Root-cause analysis (5 Whys):** look beyond the symptom to the underlying cause.

- **Energized work:** tired, stressed people are not mindful, so XP promotes a sustainable pace.

Example: a team notices every build is taking a minute longer than last month (awareness). Using 5 Whys they find a slow test suite caused by repeated database setup; they fix it in one afternoon. Without mindfulness the build would have reached 30 minutes and developers would stop running it.

Q5 [L2 | 10 marks]

Illustrate how the roles and responsibilities of Extreme Programming (XP) team members contribute to successful software development with example.

XP uses the **whole team** idea: everyone needed for the product's success sits together and shares responsibility. There are few formal job titles, but several distinct roles.

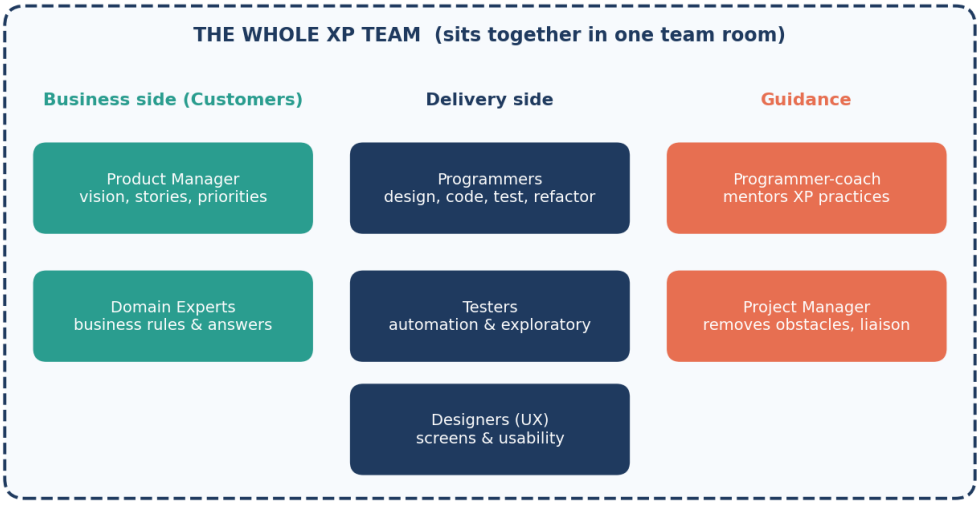


Fig. 4: Roles in an XP team

Role	Responsibilities	Contribution to success
Product manager (on-site customer)	Creates and maintains the vision; writes and prioritises stories; accepts finished work; decides release dates	Team always builds the most valuable thing; fewer wasted features
Domain experts	Provide business rules and answers to detailed questions (e.g., billing, regulations)	Correct requirements; fewer assumptions
Designers (UX / interaction)	Design screens and flows, run usability checks, work with programmers on UI stories	Product that users can actually use
Programmers	Estimate stories, design simply, write code with TDD, pair, refactor, integrate continuously, share code ownership	Working, maintainable software delivered every iteration
Testers	Help customers define acceptance examples, automate tests, do exploratory testing, find gaps early	Quality built in; defects found in hours, not months
Programmer-coach	Mentors XP technical practices and team learning	Practices stay healthy; skills grow
Project manager	Removes obstacles, manages external communication and resources, reports progress	Team stays focused and supported

Example: ride-booking app "RideEasy"

The **product manager** prioritises the story "cancel a ride and get a refund". The **domain expert** from the operations team explains the cancellation-fee rules. The **designer** sketches the cancel screen. A pair of **programmers** writes a failing test for the refund rule, then the code. The **tester** adds exploratory tests (cancel during payment, no network). The **programmer-coach** notices the refund code duplicates the wallet logic and suggests refactoring it. The **project manager** gets the payment-gateway sandbox access that was blocking the team. The feature is demoed on Friday and released. Because every role contributed at the right moment, no feature waited for another department.

Q6 [L2 | 10 marks]

List the six prerequisites for adopting Extreme Programming (XP) and explain the importance of each prerequisite in an organization with example.



Fig. 5: Six prerequisites for adopting XP

#	Prerequisite	Why it matters	Example
1	**Management support**	XP needs a team room, dedicated people, training time, tolerance for an initial slowdown and sensible metrics. Without support the team cannot practise XP.	A company provides an open team room and sends the team to a TDD workshop.
2	**Team agreement**	Practices such as pairing and collective ownership change how people work. Forced adoption leads to quiet resistance; the team must be willing to try them honestly for a few iterations.	The team votes to run XP for 12 weeks and then review.
3	**Colocated team**	Sitting together gives instant, face-to-face communication and shared awareness; questions get answered in seconds, not days.	A developer asks the tester across the table about a failing case instead of emailing.
4	**On-site customers**	Real business people (product manager, domain experts) must be available daily to clarify stories, set priorities and accept work.	A bank officer sits with the team building a loan app.
5	**Right team size**	Roughly 6-12 people (up to ~20 with all roles) works best. Very small teams lack pairing variety and skills; very large ones suffer communication overload and need to be split.	A 25-person project is split into two XP teams of 12.
6	**Use all the practices**	XP practices reinforce one another; each covers weaknesses of the others. Cherry-picking breaks the safety net.	TDD makes refactoring safe; refactoring keeps the design simple enough for pairing and shared ownership.

Remember: M-T-C-O-S-U (**M**anagement, **T**eam agreement, **C**olocated, **O**n-site customers, **S**ize, **U**se all practices).

Q7 [L2 | 10 marks]

Explain the recommendations for successful adoption of Extreme Programming (XP) in a software development organization.

Adopting XP is a change in *habits*, not only tools, so it must be planned like learning a new sport: with a coach, practice and patience.

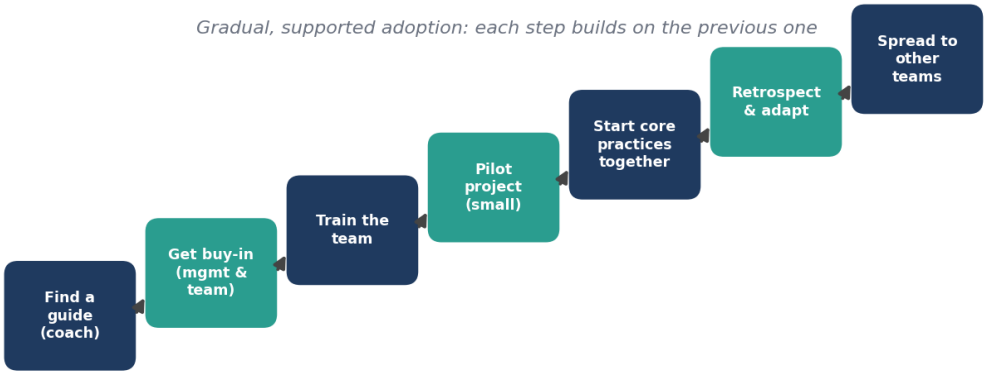


Fig. 6: A gradual path to XP adoption

1. **Find a guide.** Use an experienced XP coach or mentor for the first weeks. *Example:* an external coach pairs with the team to set up TDD and CI in the first sprint.
2. **Get management and team agreement first.** Explain benefits, risks and what you need (team room, dedicated staff, training), and obtain voluntary commitment.
3. **Train the team.** Short hands-on workshops on pairing, TDD, stories and retrospectives. Practise on a small exercise before real work.
4. **Start with a pilot project.** Choose a real but low-risk, modest-sized project and the most open-minded team. Success stories convince others better than slides.
5. **Begin with the core practices together.** Short fixed iterations, stories, stand-ups, pair programming, TDD, CI, retrospectives. Do not drop the "uncomfortable" ones.
6. **Be honest about the current state.** Measure where you are (defects, lead time, build time) so improvements are visible.
7. **Expect a dip, then gains.** Productivity often falls briefly while people learn. Management should expect this.
8. **Retrospect and adapt.** Review every iteration; adjust practices to fit your context, but only after you understand why a practice exists.
9. **Spread gradually.** Once the pilot works, let its members coach the next team.

Example: a Bengaluru product company piloted XP with one 8-person team for three months. Defects found by customers dropped, releases went from quarterly to monthly, and two programmers from the pilot became coaches for the next two teams.

Q8 [L3 | 10 marks]

Demonstrate how team size and full-time participation influence the implementation of Extreme Programming (XP) practices.

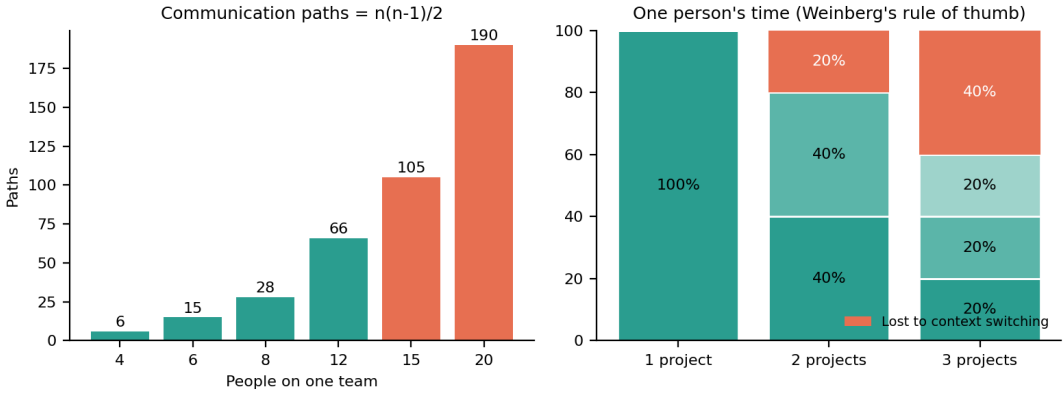


Fig. 7: Left - communication paths grow rapidly with team size. Right - multitasking wastes time (Weinberg's rule of thumb)

A. Influence of team size

Team size	Effect on XP practices
Too small (fewer than about 4 programmers)	Few pairing combinations, so pairs get stale; limited skills and review; absence of one person is a big risk; hard to rotate coaching or customer duties.
Right size (about 6-12, up to ~20 with all roles)	Stand-up still fits in 10-15 minutes; pairs rotate; everyone can know the whole codebase; room fits one team space; collective ownership works.
Too large (15+ programmers)	A team of 15 has 105 communication paths (a team of 6 has 15). Stand-ups drag, people specialise in modules, decisions slow down, pair rotation and shared ownership fail. Solution: split into two or more teams, each with its own coach and customer representative.

B. Influence of full-time participation

- **Pair programming** needs partners who are present at the same time. Part-timers cannot pair reliably.
- **Stand-ups, planning and retrospectives** lose value if some members are missing, and their knowledge never reaches the team.
- **Velocity** becomes unpredictable when people come and go, so release forecasts are unreliable.
- **Multitasking is costly.** Weinberg's rule of thumb: working on 2 projects leaves about 40% per project (20% lost to switching), and 3 projects leaves only about 20% each (40% lost).
- **On-site customers** must also be available full-time (or nearly) to answer questions quickly.

Example: Team "CampusCart" has 6 full-time members in one room. A story about discount rules is clarified, coded, tested and demoed in two days. In another company the same story took two weeks because the tester was shared with two other projects, the customer visited only on Fridays and three developers worked on separate modules.

Practical guidance: dedicate people to one XP team; if the team is large, split it; if specialists cannot be full-time, make them "on-call" experts and do not count them in velocity.

Q9 [L3 | 10 marks]

Illustrate the application of Extreme Programming (XP) practices in an ongoing software development project with a suitable example.

Project: "MediTrack", a pharmacy inventory app. The team (6 developers, 1 tester, product manager from the pharmacy chain) is in **iteration 5 of 12**. Current velocity is about 20 points. This week's story: "*Alert the pharmacist when stock of a medicine falls below its reorder level*" *(5 points).*

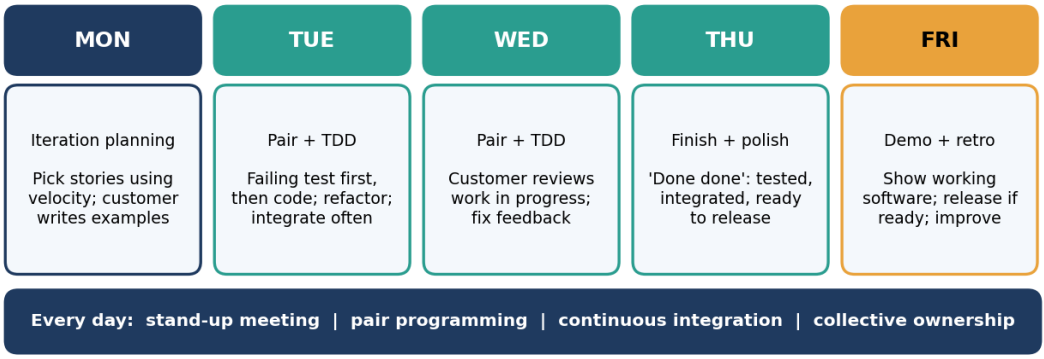


Fig. 8: A typical XP week in MediTrack

Practice	How MediTrack applies it this week
Iteration planning + stories	Monday: the product manager reads the story; the team splits it into tasks and commits to 20 points based on velocity.
On-site customer	The pharmacist-domain expert explains that expiry-date items must be excluded from the count; the answer comes in minutes.
Pair programming	Asha and Ravi pair on the alert logic; they swap the keyboard every 20-30 minutes; pairs rotate next day.
Test-driven development	They write a failing test first, then the code to pass it (see below), then clean up.
Refactoring + simple design	After the test passes they remove duplicated stock-count code shared with the "expiry alert" story.
Continuous integration + ten-minute build	Code is integrated several times a day; the CI server runs all tests; a broken build is fixed first.
Collective code ownership	Another pair finds a bug in the same module on Wednesday and fixes it without asking permission.
Stand-up	Daily 10-minute stand-up: yesterday, today, obstacles.
Demo + retrospective	Friday: the pharmacist sees the working alert and gives feedback. Retro action: "keep stories under 5 points".

TDD in one minute (Python)

```
def test_alert_when_stock_below_reorder_level():
    item = Medicine("Paracetamol", stock=8, reorder_level=10)
    assert needs_reorder(item) is True          # RED: fails first

def needs_reorder(item):                        # GREEN: simplest code that passes
    return item.stock < item.reorder_level
```

Outcome: the story is *done done* by Thursday, the demo is accepted, and the team has measured data (velocity, build status) to plan the next iteration. Because all practices work together, the project stays flexible when the pharmacy chain adds a new rule in iteration 6.

Q10 [L3 | 10 marks]

Describe the challenges of adopting Extreme Programming (XP) in different organizational environments.

XP assumes a collaborative, co-located, empowered team. Many real organisations differ, which creates challenges.

Environment	Typical challenges	Ways to handle
Large hierarchical / traditional companies	Command-and-control culture; managers uncomfortable with self-organising teams; separate departments for analysis, coding and testing	Executive sponsor; pilot team; explain XP in business terms (risk, time to market)
Regulated or government projects	Mandatory phases, documents and sign-offs; fixed scope and price; auditors expect traceability	Keep gates; run XP iterations inside phases; generate documents from stories and tests
Distributed / outsourced teams	Not co-located; time-zone gaps; customer far away; weaker informal communication	Overlap hours, video team room, remote pairing tools, customer proxy, frequent demos
Legacy codebases	No tests, duplicated code, fear of change; large defect backlog	Add tests as you touch code; refactor gradually; allocate time to pay debt
Fixed-price contracts	Contract assumes all requirements known upfront; changes seen as "extra cost"	Contract in phases or with a changeable scope; prioritised stories
Large teams / large systems	Communication overload; many dependencies	Split into XP teams with coordination meetings; shared standards
Skill and culture gaps	Developers unfamiliar with TDD/pairing; "I work best alone"; ego; no coach	Training, coaching, voluntary trial, rotating pairs
Wrong metrics and multitasking	Success measured by lines of code or utilisation; people on multiple projects	Measure delivered value, defects, cycle time; dedicate people

Common root causes: lack of management support, partial adoption of practices ("XP-lite"), unavailable customers, and expecting instant results. Because XP is a system of mutually supporting practices, organisations succeed when they treat adoption as gradual cultural change supported by coaching, pilots and measurement.

PART 2: Scenario-Based Questions

Q1 [L3 | 10 marks]

Mobile banking application: deadline pressure, hard-coded values, duplicate code, weekly payment changes touching several files, more time spent fixing old code than writing new features. (a) Identify XP concepts (b) Refactoring (c) Technical debt and consequences (d) Mindfulness.

(a) Relevant XP concepts

- **Technical debt:** shortcuts (hard-coding, duplication) taken for speed.
- **Refactoring:** the remedy: improve structure without changing behaviour.
- **Simple design / "once and only once":** no duplicate logic, no magic numbers.
- **Test-driven development and continuous integration:** the safety net that makes frequent change safe.
- **Collective code ownership and pair programming:** anyone can clean any code; pairing prevents bad shortcuts.
- **Mindfulness and retrospectives:** noticing the pain early; **sustainable pace / slack:** time to clean up.

(b) How refactoring improves the code

Refactoring changes *how* the code is organised, not *what* it does. Done in small steps with tests running after each step, it removes duplication, names constants and puts each rule in one place. Then a weekly payment change touches **one** file instead of many.

Refactoring example (fees are illustrative)

```
# BEFORE: duplicated, hard-coded (every new payment mode = copy + paste)
def upi_charge(amount):
    fee = amount * 0.005
    if fee > 20: fee = 20
    return fee + fee * 0.18

def neft_charge(amount):
    fee = amount * 0.005
    if fee > 20: fee = 20
    return fee + fee * 0.18

# AFTER: one rule table + one function (rules can come from configuration)
TAX_RATE = 0.18
FEE_RULES = {"UPI": (0.005, 20), "NEFT": (0.005, 20)}   # (rate, cap)

def charge(mode, amount):
    rate, cap = FEE_RULES[mode]
    fee = min(amount * rate, cap)
    return fee * (1 + TAX_RATE)
```

Typical refactorings here: *Extract method*, *Replace magic number with named constant / configuration*, *Remove duplicate code* and *Introduce a common interface* for payment modes. Each step is covered by automated tests so the banking behaviour stays correct.

(c) Technical debt and its consequences

The debt here: hard-coded values, duplicated code and a design where one change ripples through many files (a "shotgun surgery" smell). The team borrowed speed from the future to meet a deadline.

Fig. 9: Debt slows delivery over time (illustrative)

- **Interest:** every new feature takes longer because developers must edit and re-test many files.
- **More defects:** a change missed in one copy causes inconsistent fees or limits, a serious problem in banking.
- **Risk:** hard-coded limits or secrets are security and compliance risks.
- **Lower morale and higher cost:** developers spend their days fixing old code; delivery dates slip; eventually a costly rewrite may be needed.

Repaying it: reserve a share of every iteration (for example 15-20%) for refactoring, and follow the rule "leave the code cleaner than you found it".

(d) How mindfulness helps detect problems early

- **Notice the signals:** a one-line payment change touched six files; the same bug appears in two modules; velocity is dropping.
- **Retrospectives:** ask "why did we spend 60% of last week on rework?" and apply **5 Whys** to reach the cause (deadline shortcuts, no tests).
- **Pair programming:** the partner questions a hard-coded value at the moment it is written.
- **Visible information:** a board that tracks time spent on new features versus fixing, and a build/test dashboard, makes the debt visible to everyone.
- **Stand-ups:** developers say "this change was painful because..." and the team acts at once rather than after months.

Q2 [L2 | 10 marks]

Food delivery startup: launch in four weeks, 8 features requested, 5 more added in the final week; cannot finish all without delay. (a) Timeboxing (b) User stories for prioritisation (c) Last Responsible Moment (d) Delivery strategy.

Assumption for the numbers: the team's velocity is about 20 points per week, so four weeks give about 80 points, of which about 75-80% (about 60 points) is planned to leave slack for defects.

(a) Applying timeboxing

Timeboxing fixes **time** (launch date and four one-week iterations) and keeps **scope flexible**. The question changes from "how long will all 13 features take?" to "which features deliver the most value in these four weeks?". Each week has its own box: whatever is not *done done* moves out of the iteration instead of stretching the deadline.

(b) Using user stories to prioritise

Each feature is written as a small story with an estimate, then ranked by **business value, risk and effort** (for example with MoSCoW: Must / Should / Could / Won't now). The customer, not the programmers, decides the order.

Story	Points	Priority	Decision
1 Sign-up / login	5	Must	Release 1 (week 1)
2 Browse and search restaurants	8	Must	Release 1 (week 1)

Story	Points	Priority	Decision
3 Menu and cart	8	Must	Release 1 (week 1)
4 Place order + online payment	13	Must	Release 1 (week 2)
5 Order status (basic)	5	Must	Release 1 (week 2)
6 Order history / reorder	5	Should	Release 1 (week 3)
7 Ratings and reviews	5	Should	Release 1 (week 3)
8 Push notifications	5	Should	Release 1 (week 3)
9 Coupons / promo codes (new)	5	Should (launch marketing)	Release 1 (week 3)
10 Live map tracking (new)	13	Could	Release 1.1
11 Loyalty points (new)	8	Could	Release 1.1
12 Scheduled orders (new)	8	Could	Release 1.1
13 In-app chat support (new)	8	Won't now	Backlog

Release 1 = 59 points (fits the 4-week capacity with slack); 29 points go to Release 1.1; 8 points stay in the backlog.

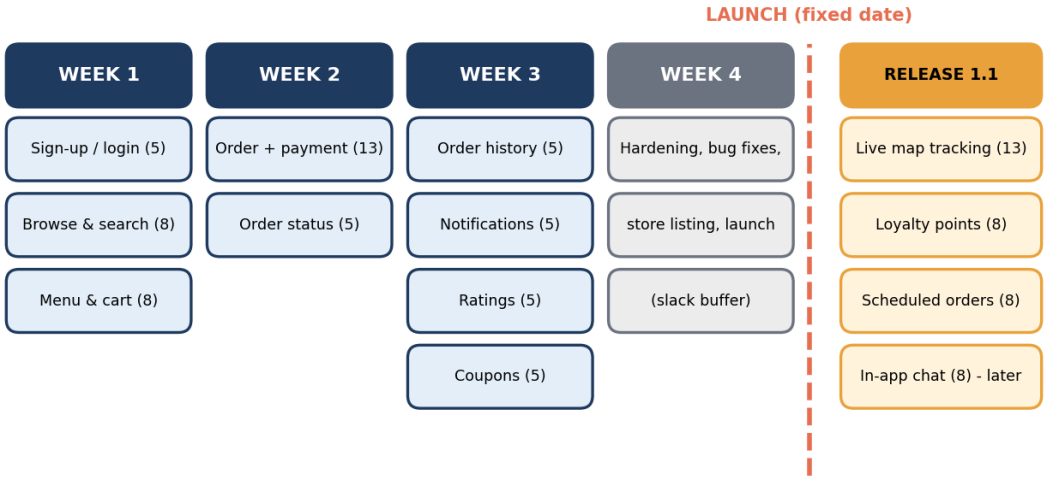


Fig. 10: Timeboxed plan: fixed launch date, flexible scope

(c) Last Responsible Moment (LRM)

Delay decisions about **optional** features until the latest moment at which waiting would hurt, so they are made with real usage data.

- **Live map tracking:** do not choose a maps provider or design the tracking architecture now. After launch, real order data shows whether customers want it and which provider is affordable.
- **Loyalty points:** decide the earn/burn rules after seeing repeat-order behaviour.
- **But do not delay blockers:** the payment gateway and database choice must be made in week 1 because other stories depend on them.

Keeping the design simple and well tested makes later additions cheap, which is what makes late decisions safe.

(d) Recommended delivery strategy

1. Launch a **minimum viable product** (Release 1) on the fixed date containing all Must and chosen Should stories.
2. Adopt a **swap rule**: a new request enters the current iteration only if an equal-size story leaves; otherwise it goes to the backlog.
3. Schedule **Release 1.1** two weeks after launch for the Could stories.

4. Demo every Friday so the client sees progress and re-prioritises early, not at the last minute.
5. Use feature switches so partially built features never block the launch.

Q3 [L3 | 10 marks]

Team of eight adopts XP but three developers refuse pair programming, testers are excluded from planning and refactoring is skipped; integration failures and defects increase. (a) Prerequisites/practices violated (b) Team agreement (c) Practices together (d) Strategy.

(a) Prerequisites and practices being violated

Violated item	Evidence in the scenario
Team agreement	Three developers refuse to take part: the team has not truly agreed to try XP.
Use all the practices	Pair programming and refactoring are skipped; likely also weak TDD and CI, as shown by integration failures.
Whole team	Testers are excluded from planning, so quality concerns and testability are not considered.
Management/coach support (probable)	No one is guiding the practices or resolving the conflict.

The team size (8) is fine; the problem is attitude and incomplete practice.

(b) Importance of team agreement

- XP practices are **social**: pairing, shared ownership and open planning only work if people genuinely cooperate.
- Forced practices produce "malicious compliance" or silent resistance; the three refusers weaken the other five.
- Agreement creates shared standards (definition of done, coding standard, working hours) and accountability.
- It does not need unanimous enthusiasm; it needs willingness to **try honestly for a fixed time** and judge by results.

(c) Why practices must work together

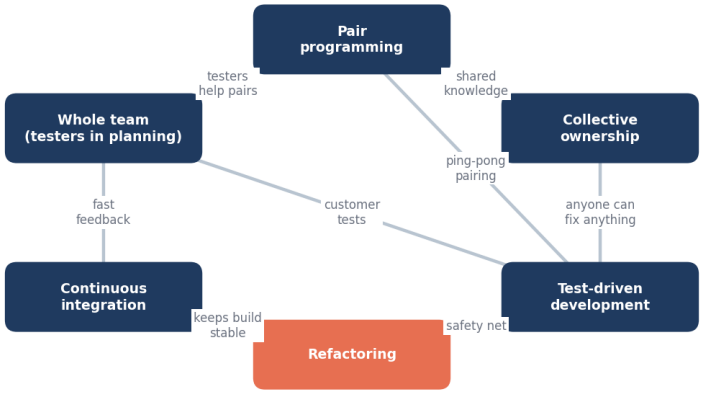


Fig. 11: XP practices support each other

- **TDD** provides the safety net that makes **refactoring** safe; without refactoring, the design rots and tests become hard to write.
- **Pair programming** spreads knowledge, so **collective ownership** is possible; without pairing, only the author understands the code.

- **Continuous integration** works only if small, tested changes are committed often; skipping tests means the build breaks.
- **Testers in planning** bring acceptance examples that guide TDD and prevent misunderstandings.

Choosing practices individually leaves gaps. **Example:** removing the seat belt while keeping the airbag still leaves you exposed.

(d) Practical strategy

1. **Facilitated retrospective:** ask the three developers why they resist (slower? discomfort? ego?) and listen.
2. **Working agreement:** the team defines core hours, pairing rules, definition of done, a broken-build rule.
3. **Time-boxed experiment:** try pairing and refactoring for two iterations, then compare defects and integration failures with the baseline.
4. **Make pairing comfortable:** pair on hard tasks first, let people choose partners, rotate pairs every day or two, and let them work solo on trivial tasks with a review.
5. **Include testers in planning** and let them write acceptance examples before development starts.
6. **Build refactoring into "done":** every story includes cleaning the touched code; reserve slack for bigger clean-ups.
7. **Automate integration:** CI with fast tests; fix a broken build immediately.
8. **Track results** (defects, build failures, rework time) on a visible board and review them in retrospectives. If problems persist, involve the coach or management.

Q4 [L3 | 10 marks]

Hospital management system: six programmers in different cities; administrator available once a week; incorrect assumptions about appointment scheduling and billing; rework rising. (a) Prerequisites not satisfied (b) Co-located team and on-site customers (c) Customer proxy (d) Recommendations.

(a) Prerequisites not adequately satisfied

Prerequisite	Status	Reason
Colocated team	**Not satisfied**	Programmers are in different cities.
On-site customers	**Not satisfied**	The administrator is available once a week only.
Right team size	Satisfied	Six programmers is a good size.
Management support / team agreement	Needs checking	Support for travel, communication tools and a customer proxy is needed.

(b) Importance of a co-located team and on-site customers

- **Co-located team:** face-to-face talk is the fastest, richest communication. People overhear each other, so errors in understanding are caught at once; pairing is simpler; trust grows.
- **On-site customers:** domain rules (who can book which doctor, how insurance splits a bill) are complicated and cannot be guessed. When a customer is present, questions are answered in minutes, priorities are corrected early and finished work is accepted daily.
- **Cost of their absence (this scenario):** wrong assumptions are coded, found a week later and reworked. A one-week feedback loop means each misunderstanding wastes days.

Fig. 12: Before and after: shorter feedback loop with a customer proxy

(c) How a suitable customer proxy helps

A **customer proxy** is a person who represents the real customer to the team when the real customer cannot be present. In the hospital: a business analyst or the **billing supervisor / front-desk head** who knows scheduling and billing well.

- Answers questions daily, so the team does not wait a week.
- Writes stories and concrete examples (for example "Dr. Rao has 15-minute slots; emergency patients bypass the queue").
- Sits in planning and demos, and takes the team's questions to the administrator in one batch.
- **Needs:** real domain knowledge, access to the administrator and actual staff, authority to decide small things, and time to be available. A weak proxy only adds another "telephone game" layer.

(d) Recommendations to reduce delays and incorrect requirements

1. **Co-locate if possible** (a hub office); otherwise create a **virtual team room**: always-on video, chat, shared board and remote pair-programming tools.
2. Fix **core overlap hours** (for example 10:00-14:00 IST) when everyone is online; hold a daily video stand-up.
3. Appoint a **customer proxy** and ask the administrator for at least one or two scheduled slots weekly, plus a quick-answer channel.
4. Capture rules as **examples and acceptance tests** (appointment, cancellation, billing scenarios) reviewed by the proxy before coding.
5. Use **short iterations and weekly demos** (recorded if needed) so errors are found within days.
6. Adopt the habit "**ask early, never assume**": unclear rule means a flagged question on the shared board.
7. Track rework in retrospectives to show management the cost of poor communication.

Q5 [L3 | 10 marks]

A team plans 30 points but completes 16, then 18 in the next iteration. Developers finish coding early, testers get large batches at the end; defects delay releases. (a) Average velocity (b) Using velocity (c) Constraint (d) Testing and workflow changes.

(a) Average velocity

Velocity counts only **completed (done done)** points. The planned 30 is not velocity.

Average velocity = (16 + 18) / 2 = 17 story points per iteration.

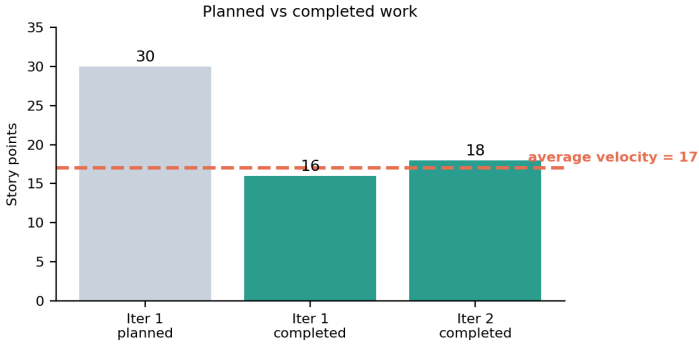


Fig. 13: Planned versus completed points

(b) How velocity should be used in future planning

- **Plan the next iteration at about 17 points** (not 30). Use the average of the last 3 iterations once available.
- **Forecast releases:** if 85 points remain, 85 / 17 = **5 iterations**; using the range 16-18 gives about 5-6 iterations (85/18 = 4.7, 85/16 = 5.3).
- Do **not** pad the plan to "catch up", do not count partial stories, and do not compare velocity between teams or use it as a performance target.
- Recalculate every iteration; velocity should stabilise as the team learns.

(c) The likely constraint and its effect on throughput

Testing is the constraint (bottleneck). Developers are fast, but work waits in a queue for testers, who receive large batches at the end of the iteration. A story counts only when tested, so **throughput equals tester capacity (about 16-18 points)**, however fast programmers code.

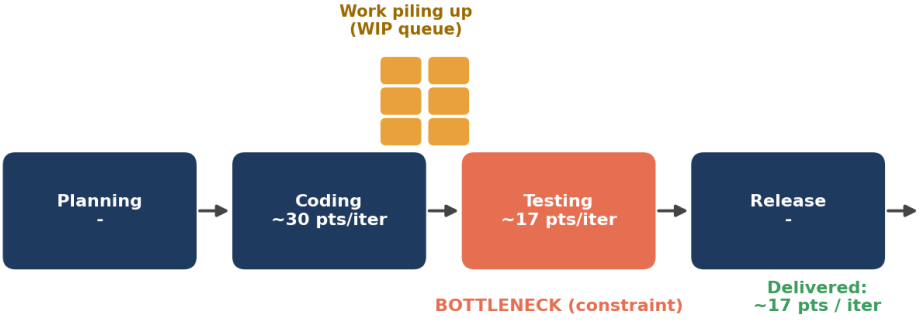


Fig. 14: Testing limits throughput; coding faster only increases the queue

- Untested stories pile up (WIP), so many remain unfinished at iteration end and velocity is low.
- Defects are found days after the code was written, so fixes cost more.
- Releases are delayed by unresolved defects.

(d) Recommended changes (apply the five focusing steps)

Step	Action
Identify	Testing capacity is the constraint, confirmed by queue length and cycle time.
Exploit	Keep testers busy on valuable work: automate repetitive regression tests, remove meetings and waiting.
Subordinate	Programmers stop starting new stories when the test queue is full; they help by writing automated tests, fixing defects and paying down debt (**limit work in progress**).
Elevate	Pair testers with programmers; adopt TDD and customer tests; add testers or automation; use continuous integration.
Repeat	Re-measure; the next constraint may be customer feedback or deployment.

- **Hand over work continuously** (each story as soon as coded), not in a batch at the end.
- Make stories **small** (about 1-3 days) so they flow smoothly.
- Include testers in planning and have them write acceptance examples **before** coding.
- Apply a strict **"done done"** definition and the "no bugs" rule: fix defects before starting new work.

Q6 [L3 | 10 marks]

Organisation adopts XP but management refuses a common workspace, allocates developers to three projects, measures success by lines of code, and gives no training. (a) Missing prerequisites (b) Importance of management support (c) Communicating benefits and risks (d) Changes.

(a) Missing prerequisites

- **Management support:** no workspace, no training, wrong metrics, no interest in the team's needs.
- **Colocated team:** no common workspace means no pairing space or shared awareness.
- **Right team / full-time participation:** developers split across three projects cannot form a stable XP team.
- **Use all the practices:** pairing and automated testing are expected without training or tooling, so they cannot be practised properly.
- **Team agreement:** not mentioned; it must be confirmed once support exists.

(b) Why management support is important

- Management controls **resources**: team room, tools, CI server, training time and people's assignments.
- XP changes how work is measured and organised; without management approval the old rules (utilisation, lines of code) override XP behaviour.
- Management must accept an **initial dip** in speed while the team learns and must shield the team from interruptions.
- Without support, XP becomes "XP in name only", and the failure is blamed on the method.

(c) Communicating benefits and risks to management

1. **Use business language:** fewer production defects, faster time-to-market, lower rework cost, predictable delivery and happier customers, not jargon.
2. **Propose a small pilot** with clear start and end dates and success measures, so management risks little.
3. **Show evidence:** case studies, and after the pilot your own data (velocity, defects, demos).
4. **Be honest about risks:** temporary slowdown, training cost, need for customer time and cultural change, each with a mitigation.
5. **Invite managers to demos** and share simple visible reports so they see working software every week.
6. **Ask for specific commitments:** workspace, dedicated staff, training budget, new metrics.

(d) Suggested changes to working arrangements

Current situation	Recommended change
No common workspace	Open team room with pairing workstations, whiteboards and visible task / build boards
Developers on three projects	Assign each developer to **one** XP team (multitasking can waste 20-40% of time)
Success measured by lines of code	Measure delivered value: stories completed, escaped defects, cycle time, customer satisfaction. More code means more maintenance; good XP often deletes code
No training or support	Hands-on training in pairing, TDD and CI; an in-house or external coach; time to build test automation
Expectation of immediate speed	Agree a learning period and review progress at 3 months

Q7 [L3 | 10 marks]

15 programmers in a newly formed XP team; no programmer-coach; project manager makes technical decisions; several programmers work alone on separate modules; a resignation leaves a module nobody understands. (a) Problems (b) Roles (c) Pairing and knowledge sharing (d) Team structure.

(a) Problems caused by team size and working arrangement

- **Team too large:** 15 programmers have 105 communication paths; stand-ups and planning become long; shared ownership and pair rotation break down.
- **No programmer-coach:** nobody mentors the practices, so XP drifts back to old habits.
- **Project manager decides technically:** slow decisions, ignores the team's expertise, discourages ownership.
- **Solo module ownership:** a "bus factor" of one: knowledge is locked in individuals. The resignation shows the risk; the module is unfinished, unmaintainable and delays the project.

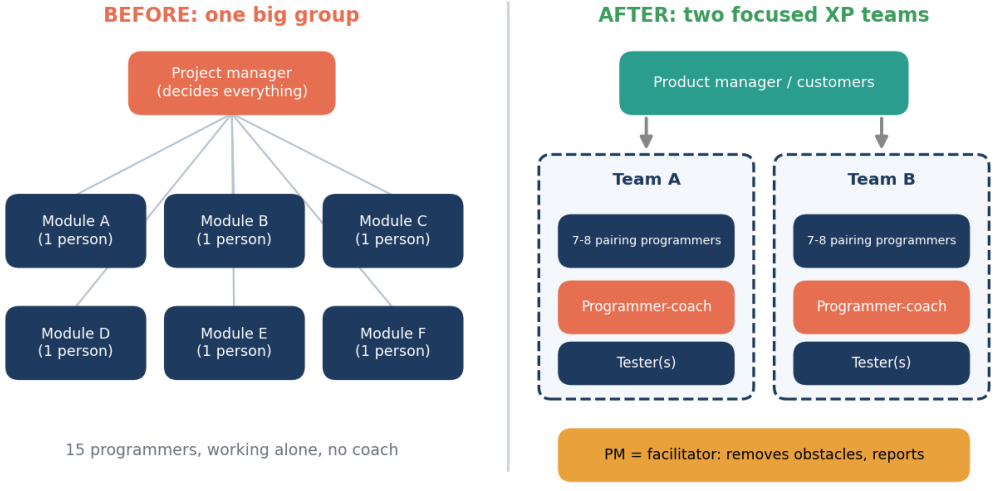


Fig. 15: From one large group of individuals to two supported XP teams

(b) Roles of the programmer-coach and project manager

Role	Should do	Should not do
Programmer-coach	Mentor pairing, TDD, refactoring and design; guide technical discussions; help the team improve; build skills so the team becomes self-sufficient	Dictate designs or work alone on the hard parts
Project manager	Remove obstacles; handle resources, external communication and reporting; support customers and the team; facilitate, not control	Make technical decisions or assign individual programming tasks

Technical decisions belong to the programmers, guided by the coach and tested through the shared code and tests.

(c) How pair programming and knowledge sharing reduce risk

- Every piece of code is understood by **at least two people**, so one resignation or illness does not stop progress.
- **Rotating pairs** and **collective ownership** spread knowledge of every module across the team.
- Continuous review while coding catches defects early and spreads good practices.
- **Tests and simple code** act as living documentation; new joiners become productive faster.
- In the scenario, if the unfinished module had been developed in pairs with tests, the remaining programmer could have continued it on the next day.

(d) Recommended team structure for introducing XP

1. **Split into two teams** of 7-8 programmers (each with 1-2 testers) that share a single product manager or have one customer representative each.
2. Provide a **programmer-coach** for each team (hire or borrow one experienced person; an external mentor at first).
3. Start with **one pilot team** if resources are limited, and let the second team adopt XP after learning from the first.
4. Assign work by **stories** (features), not by individual modules; pairs rotate across modules.
5. Redefine the project manager as a **facilitator and organisational coach**.
6. Keep the teams in the same area and hold a short cross-team meeting for dependencies.

Q8 [L3 | 10 marks]

Ten-year-old student information system: duplicated code, inadequate tests, large defect backlog. Management wants XP and all new features immediately. (a) Challenges (b) Steps to bring order (c) Prioritising debt, backlog and bugs (d) Progressive testing and refactoring.

(a) Challenges of introducing XP into an existing codebase

- **No safety net:** without automated tests, refactoring is risky and TDD is hard to start.
- **High technical debt:** duplicated code means a single change must be made in many places.
- **Defect backlog:** the team is constantly firefighting and has no capacity for new practices.
- **Unrealistic expectations:** management expects all new features at once, ignoring the cost of cleaning up.
- **Knowledge gaps and habits:** original authors may be gone; people fear touching working but fragile code.
- The system must keep running for students, so a total stop or a big-bang rewrite is not acceptable.

(b) Steps for bringing order to the project

Fig. 16: Path from chaos to a steady XP rhythm

1. **Set honest expectations** with management: first weeks focus on stability; show a plan with visible milestones.
2. **Get team agreement** and start the basic rhythm: one-week iterations, stand-ups, retrospectives, a visible board.
3. **Put the basics in place:** version control, a one-command automated build, a CI server and a bug tracker.
4. **Triage the defect backlog** and technical-debt list with the customer; close stale items.
5. **Build a safety net** by writing tests around critical flows (admission, fee, results) and around any code you must change.
6. **Refactor and deliver features together:** allocate a fixed share of each iteration (for example 20-30%) to debt and bugs, and the rest to new stories.

(c) Prioritising technical debt, backlog and important bugs

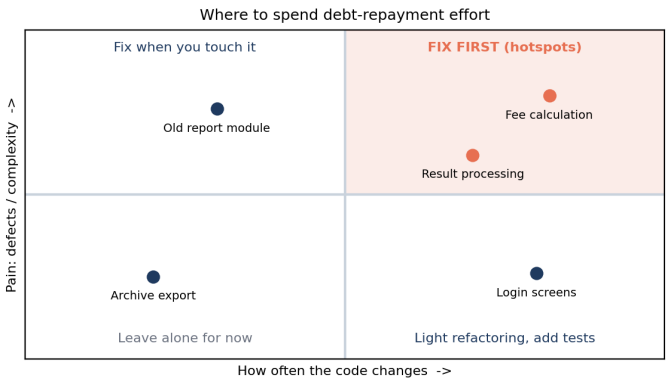


Fig. 17: Pay down debt first where code changes often and hurts most

Order	Item	Reason
1	**Critical bugs** (wrong grades, fee errors, data loss, security)	Direct harm to students and institution; fix immediately with a test that proves the fix
2	**Debt in hotspots** (high change frequency and high pain) and code needed for upcoming stories	Highest return; clean it before building new features on it
3	**Backlog review** with the customer: merge duplicates, drop stale items, convert valuable ones into stories	Shrinks the list to what truly matters
4	**Low-impact debt** in rarely changed areas	Refactor only when touched ("boy-scout rule")

(d) Introducing testing and refactoring progressively

- **Test what you touch:** for every bug fix or story, first write a failing test, then change the code; coverage grows where it matters.
- Start with **characterization / end-to-end tests** that record current behaviour of important flows, then add unit tests under them.
- Use a **coverage ratchet** in CI: coverage must never go down.
- **Refactor in small, test-protected steps** (rename, extract method, remove duplicates). Avoid a big-bang rewrite.
- Create new features in **clean modules** behind clear interfaces and gradually route old code through them (strangler approach).
- Share progress in retrospectives, using defect counts and build/test times to justify the time spent.

Q9 [L3 | 10 marks]

New e-commerce platform: management supports XP, six programmers; but no agreed practices, limited design experience, no programmer-coach, poor developer-tester relationships, unsure how to handle changing requirements. (a) Readiness (b) Prerequisites satisfied vs needing attention (c) Actions (d) Adoption plan.

(a) Evaluation of readiness

The team is **partially ready**. The most important enabler (management support) and a good team size exist, but the people side (agreement, coaching, collaboration, skills) is weak. XP can be adopted successfully *if* these gaps are closed first. Changing requirements are actually a reason to choose XP, because iterations and stories are designed for change.

(b) Prerequisites: satisfied versus needing attention

Prerequisite	Status	Comment
Management support	**Satisfied**	Keep management involved; ask for room, training and coach budget.
Right team size	**Satisfied**	Six programmers is ideal, plus testers and customer.
Team agreement	**Needs attention**	No agreed working practices yet.
Colocated team	**To verify**	Not stated; arrange one team space.
On-site customers	**To verify**	Name a product manager and domain experts.
Use all the practices	**At risk**	Limited design skills, no coach and poor tester relationship will block TDD, pairing and refactoring.

(c) Recommended actions

- **Design skills:** training on simple design and refactoring; design workshops and sketches at the whiteboard; **spike solutions** for unknowns; pair juniors with the most design-experienced developers; bring in an architect-mentor temporarily.
- **Coaching:** hire or borrow an experienced XP programmer-coach for the first 3 months, and grow an internal coach from the team.
- **Team cohesion:** kick-off workshop to create shared goals; make testers part of the team (planning, pairing with developers, shared "done done"); use blameless retrospectives; informal team activities.

(d) Adoption plan

Fig. 18: Adoption plan for the e-commerce team

1. **Working agreements** (week 0-2): core hours, pair rotation each day, broken build fixed first, no code without tests, definition of done, respectful communication; reviewed at each retrospective.
2. **Customer involvement:** a product manager sits with the team, writes stories, attends planning and demos, and answers questions daily; domain experts (payments, logistics) join when needed.
3. **Iterations:** one-week iterations from week 3; plan with velocity; start with 3-4 small stories (browse catalogue, cart, checkout).
4. **Handling change:** new or changed requirements are written as stories and re-ranked at each iteration planning; the current iteration is protected.
5. **Regular demonstrations:** weekly demo of working software to stakeholders; gather feedback; adjust the backlog.
6. **Review at 3 months:** compare defect rate, delivery predictability and team satisfaction with the baseline, then decide how to improve.

Q10 [L3 | 10 marks]

Government project with mandatory phases; management interested in XP but cannot remove phases; developers propose CI and TDD; manager worries about conflicts. (a) Challenges (b) Introducing XP within phases (c) Benefits of iterations, retrospectives, CI, TDD (d) Evaluating results.

(a) Challenges in this organisation

- **Formal phase gates and sign-offs** assume requirements and design are complete before coding, while XP evolves them incrementally.
- **Documentation and audit requirements** expect traceability and approved documents.
- **Fixed scope and contracts** leave little room to change priorities.
- **Separate specialist groups** (analysts, designers, testers) hinder the whole-team idea; customers may not be available.
- **Risk-averse culture:** the project manager fears that new practices will conflict with the process and be blamed for delays.

(b) Introducing XP within the existing phases

Fig. 19: XP inside a phased process

1. **Keep the phases and gates;** change how the work inside them is done, and make this visible to management.
2. **Start with developer-level practices that need no process change:** version control, TDD, CI, pair programming, refactoring, coding standard. They fit inside the coding and testing phases.
3. **Analysis and design:** write requirements as stories with acceptance examples; produce the required documents from them; design incrementally while still delivering the gate documents.
4. **Coding and testing:** run one-week iterations with a stand-up, a demo and a retrospective. Coding and testing overlap through TDD and customer tests.
5. **Evidence for audits:** CI logs, automated test reports and demo records serve as traceable proof of quality at each gate.
6. **Expand gradually:** add on-site customer involvement and iteration planning as trust grows.

Response to the manager's worry: CI and TDD do not conflict with the process; they produce the test evidence and early integration that the testing and deployment phases require.

(c) Benefits

Practice	Benefit in this project
Iterations	Working software every week; early feedback reduces the risk of building the wrong thing; progress is visible to auditors and stakeholders.
Retrospectives	Continuous improvement of the process within the rules; problems surface early; the team learns to adapt.
Continuous integration	Integration problems are found within minutes, not at the end; a working build always exists; deployment is smoother.
Test-driven development	Fewer defects; executable specification; safety net for change; a shorter and more reliable testing phase; documented test evidence.

(d) Evaluating results before expanding

1. **Run a pilot** on one low-risk module or subsystem for a fixed period (for example 3 months / 6 iterations) with formal approval.
2. **Set a baseline** from earlier projects and compare: defect density and defects found in UAT, rework percentage, lead time per feature, planned versus delivered work, build success rate, test coverage.
3. Collect **stakeholder and team feedback** (satisfaction surveys, retrospective notes).
4. **Review at a phase gate** with management; decide to continue, adapt or stop.
5. Document lessons, then **expand gradually** to other teams with a coach from the pilot team.

Quick Revision Sheet

Topic	One-line memory hook
XP life cycle	Vision, release planning, repeating 1-week iterations (plan, daily dev, demo, retro), release, feedback
Coaches	Programmer-coach (technical) + Project manager (organisational); both enable, not control
Key concepts	Stories, Iterations, Velocity, Timeboxing, Last Responsible Moment, Refactoring, Technical debt
Theory of Constraints	Identify, Exploit, Subordinate, Elevate, Repeat: speed = slowest step
Mindfulness	Pay attention (pairing, stand-ups, retros, 5 Whys), then improve
Roles	Product manager, domain experts, designers, programmers, testers, programmer-coach, project manager
Six prerequisites	Management support, Team agreement, Colocated, On-site customers, Right size, Use all practices
Velocity	Completed points per iteration; average = sum / iterations; plan with it, never with wishes
Technical debt	Shortcut now = interest later; repay with refactoring, tests and slack
Legacy systems	Triage, safety-net tests where you touch, refactor in small steps, no big-bang rewrite
Phased organisations	Keep gates, run XP inside; use tests and CI as audit evidence; pilot then expand

Module 3 Answer Key: Practicing XP

10 questions × 10 marks CO3 · BTL L2 (Understand) Source: class notes "Module 3 – Practicing XP"

Scope check: the brief mentioned DBMS topics (SQL, views, triggers, transactions), but both uploaded files are for **Agile Technology, Module 3 (Practicing XP)**. This key follows the uploaded question bank and notes. Q8, Q9 and Q10 are only listed by name in the notes, so they are completed from *The Art of Agile Development* (Shore & Warden), the notes' stated reference.

---

1 **Explain Pair Programming in XP. Discuss its benefits, working mechanism, and how it improves software quality, with an example.**

CO3 L2 10 marks

Definition: Pair programming is an XP practice (Theme: Thinking) where **two programmers work together at one workstation** on the same task. One types while the other reviews continuously, and they swap roles often.

WORKING MECHANISM

Role	Responsibility
Driver	Tactical work: types the code, focuses on the current line or method.
Navigator	Strategic work: watches design, tests, risks, edge cases and the big picture.
Rotation	Swap driver/navigator every 30 to 60 minutes; change partners regularly to spread knowledge.

1. Pick a story or task from the iteration board.
2. Sit side by side at one machine (shared keyboard and screen).
3. Driver codes while navigator reviews, thinks ahead and asks "why?".
4. Switch roles; switch partners at the next task.

BENEFITS

- **Fewer defects:** every line is reviewed the moment it is written.
- **Knowledge sharing:** no one is the only person who knows a module.
- **Better design:** two minds discuss alternatives before committing.
- **Focus and discipline:** partners keep each other on TDD, standards and the task.
- **Mentoring:** juniors learn quickly; team cohesion improves.

HOW IT IMPROVES QUALITY

Pairing acts as **continuous code review**. It reduces rework, enforces coding standards, catches mistakes early (cheapest to fix) and keeps design simple. It works best with an even number of programmers.

EXAMPLE AND CODE

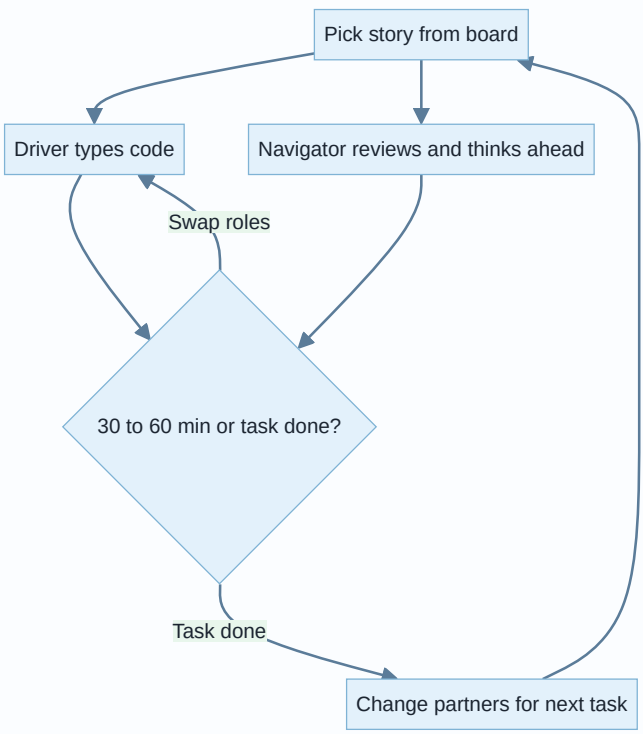
College Student App, Login story. Ravi (driver) writes a test for wrong passwords. Meena (navigator) notices that account lockout is missing and the empty-password case is untested. They add both, then swap roles.

```
# Driver writes the test (navigator suggests the extra cases)
def test_login_rejects_wrong_password():
    assert login("ravi", "wrong") is False

def test_login_rejects_empty_password():      # navigator's idea
    assert login("ravi", "") is False

def test_account_locks_after_3_failures():    # navigator's idea
    for _ in range(3): login("ravi", "bad")
    assert is_locked("ravi") is True
```

DIAGRAM: PAIRING CYCLE



Remember: Driver = hands (tactics), Navigator = eyes (strategy), Rotate = knowledge spreads.

2 **Explain Root Cause Analysis in Agile. How can a team identify the underlying causes of recurring problems and implement corrective actions? Give an example.**

CO3 L2 **10 marks**

Definition: Root Cause Analysis (RCA) is the practice of tracing a mistake back to the **underlying cause** and fixing that cause, so the problem does not return. Core rule: **blame the process, not people.**

MAIN POINTS

- **Ask "Why?" five times:** each answer becomes the next question until a fixable cause appears.
- **Blameless fact finding:** rely on observations and data, not opinions.
- **System fixes:** improve the process instead of treating symptoms.
- **When not to fix:** if fixing the root cause adds more overhead to the process than the problem costs.

STEPS FOR THE TEAM

1. **Fix the symptom** first (stabilise, restore service).
2. **Gather facts:** timeline, logs, who/what/when (no blame).
3. **Ask Why 5 times** as a group (often in a retrospective).
4. **Pick a root cause** that the team can actually change.
5. **Corrective action** (fix now) plus **preventive action** (stop recurrence).
6. **Verify** in the next iteration; inspect if the problem repeats.

EXAMPLE: FOOD DELIVERY APP, WRONG ORDER TOTAL SHOWN

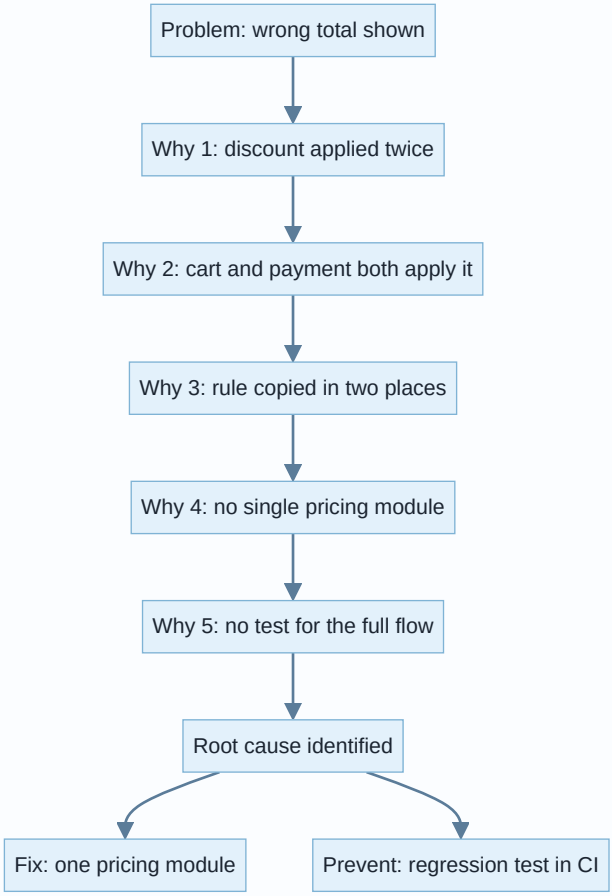
Why?	Answer
1. Why did customers see a wrong total?	Discount was applied twice.
2. Why twice?	Both the cart module and the payment module applied it.
3. Why did both apply it?	The discount rule was copied into two places.
4. Why was it copied?	No single owner or shared module for pricing rules.
5. Why was it not caught?	No automated test covered the "discount + payment" flow.

Root cause: duplicated rule and missing test. **Actions:** move pricing into one module (refactor), add a regression test, make the test part of the build.

```
def test_discount_applied_once():
    cart = Cart(items=[Item("Biryani", 200)], coupon="SAVE10")
    assert checkout(cart).total == 180      # 200 - 10%, not 160

# CI rule: build fails if this test fails  -> prevents recurrence
```

DIAGRAM: CHAIN OF WHYS



3 **What is a Retrospective in Agile? Explain its purpose, process, and role in continuous improvement, with an example.**

`CO3` `L2` `10 marks`

Definition: A retrospective is a **regular team meeting to inspect work habits** and decide small improvements. It keeps the process matched to the team's changing situation.

PURPOSE

- **Regular inspection:** look at how the team works, not only what it built.
- **Small experiments:** turn observations into actionable trials.
- **Normal work:** make process improvement routine, not a special event.

TYPES OF RETROSPECTIVES

Type	When
Iteration	End of every iteration (most common).
Release	After a release goes out.

Project	At project end; lessons for future teams.
Surprise	When an unexpected event changes the situation.

PROCESS (5 STEPS)

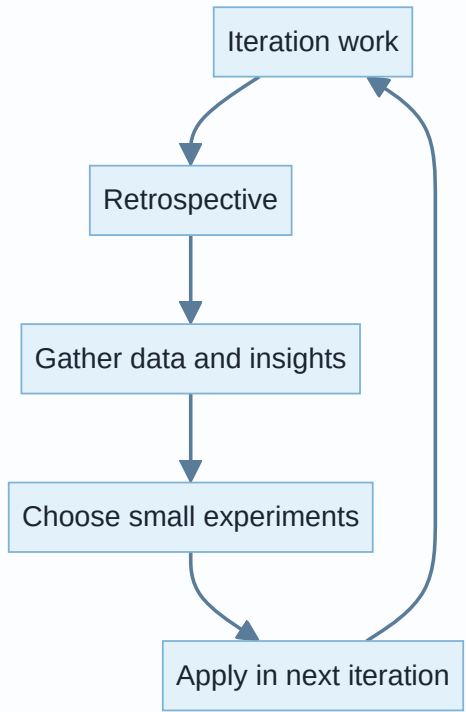
1. **Set the stage:** whole team attends, safe and blameless tone, timebox 1 hour.
2. **Gather data:** what happened (velocity, bugs, events, feelings).
3. **Generate insights:** find causes (use Root-Cause Analysis).
4. **Decide actions:** pick 1 to 3 small, owned experiments.
5. **Close:** record actions on the team board; review them next retro.

EXAMPLE: KEEP / PROBLEM / TRY, ITERATION 2 OF THE LIBRARY APP

```
KEEP     Daily stand-ups; pairing on the QR-issue story
PROBLEM  Builds take 25 minutes; two stories were "almost done" at demo
TRY      Split slow tests, target 10-minute build (owner: Arjun)
         Apply "Done Done" checklist before moving a card to Done
```

Role in continuous improvement: actions from this retro are checked in the next one, so each iteration the team's process gets slightly better (Plan, Do, Inspect, Adapt).

DIAGRAM: IMPROVEMENT LOOP



4 **Explain the importance of Trust in an Agile team. How does trust promote transparency, shared responsibility, and effective collaboration?**

CO3 L2 10 marks

Definition: Trust is the confidence that teammates will help, be honest and take **joint responsibility** for the work. For maximum productivity, members must rely on each other. To improve trust, foster **empathy**.

WHY TRUST MATTERS (THREE PILLARS)

Pillar	Meaning	Effect
Foundation	Enables honest communication.	Effective collaboration; people ask for help early.
Autonomy	Room to decide responsibly.	Shared responsibility; no micromanagement.
Transparency	Problems made visible early.	Risks are fixed while still cheap.

HOW TO BUILD TRUST

Within the team:

- Customer-programmer empathy
- Programmer-tester empathy
- Eat together
- Team continuity (keep the same team)

With the organization and stakeholders:

- Show some hustle
- Deliver on commitments
- Manage problems
- Respect customer goals
- Promote the team
- Be honest

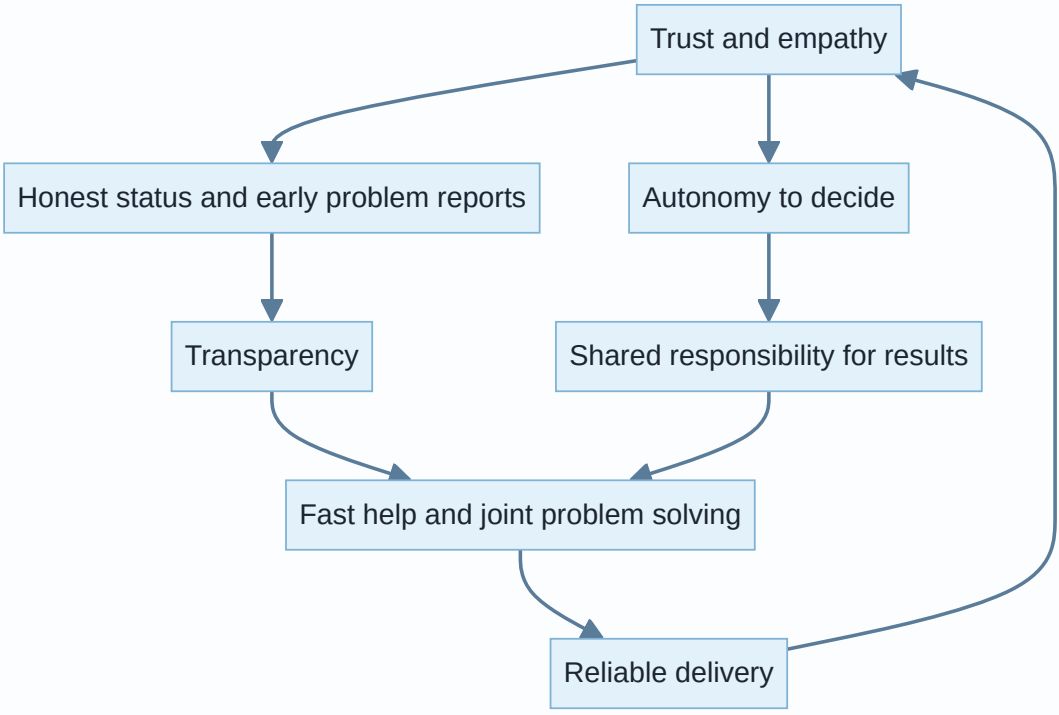
If stakeholders do not trust the team, they will not participate and may even campaign against the project.

EXAMPLE

On day 2 of an iteration, a developer says: "The payment API is harder than estimated; I need a pair." Because the team trusts each other, no one blames him. Two people swarm on it, the customer is told early and drops a low-value story. In a low-trust team the delay would be hidden until the demo.

```
Low trust : hide delay  -> surprise at demo -> blame -> less honesty
High trust: report early -> team helps       -> scope adjusted -> delivery on time
```

DIAGRAM: TRUST LOOP



5 **What is Ubiquitous Language? Explain how a common vocabulary shared by customers, developers and testers reduces misunderstandings, with an example.**

`CO3` `L2` `10 marks`

Definition: Ubiquitous Language is a **shared vocabulary taken from the customer's domain** and used by everyone (in conversation, user stories, tests and code), so no mental translation is needed between business and software.

MAIN POINTS

- **Speak the domain experts' language,** not technical jargon, to avoid miscommunication, delays and errors.
- **Shared vocabulary:** exact same terms in talk, tests and code.
- **Resolve ambiguity:** stops different groups inventing their own words for one idea.
- **Code bridge:** class and method names mirror how users talk about their work.
- **Domain model:** a powerful way to build the language (a simple diagram or glossary of key terms).
- **Living language:** domain experts influence code, and issues found in code influence the experts.

HOW IT REDUCES MISUNDERSTANDINGS

Without common language	With ubiquitous language
Customer says "registration", developer codes `Booking`, tester tests "enrolment".	Everyone says **Enrolment** everywhere.

Requirements are mentally translated; errors creep in.	No translation; rules map directly to code.
Long clarification meetings.	Faster, shorter discussions.

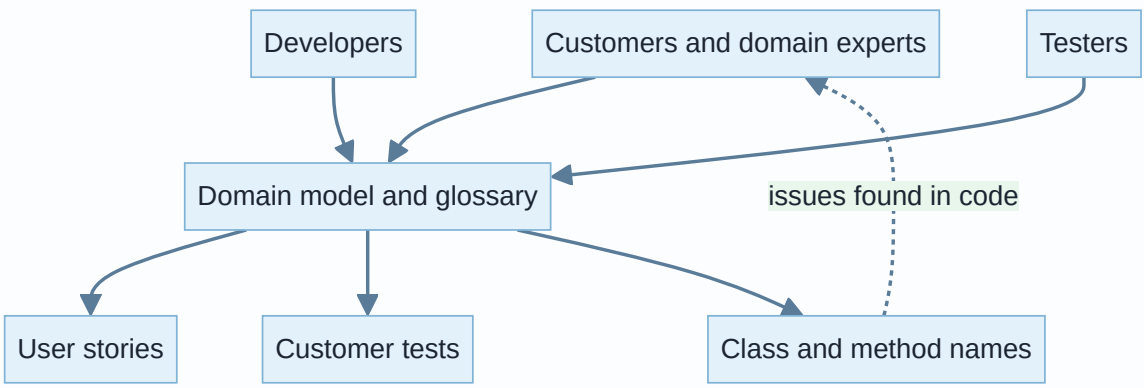
EXAMPLE: UNIVERSITY SCHEMA

Domain experts say: "A **student enrols** in a **course offering**; enrolment is rejected if the **seat limit** is reached." The same words appear in the story, the test and the code.

```
Story : As a student, I want to enrol in a course offering ...
Test  : Given a course offering with seat limit 60 and 60 enrolments
        When a student tries to enrol
        Then the enrolment is rejected

class CourseOffering:
    def enrol(self, student):
        if self.enrolments >= self.seat_limit:
            raise SeatLimitReached()
```

DIAGRAM: ONE LANGUAGE, MANY ARTIFACTS



6 **What is an Iteration Demo? Explain its procedure and significance in demonstrating working software, gathering stakeholder feedback and planning subsequent iterations.**

CO3 L2 **10 marks**

Definition: An iteration demo is a short meeting at the end of each iteration (weekly in XP) where the team **shows real, working software to stakeholders** and asks for feedback. It keeps the team's work aligned with stakeholder goals.

PROCEDURE

1. **Prepare:** demo only stories that are "Done Done"; run the demo on a real build.
2. **Invite** stakeholders (users, managers, executive sponsor) at a fixed day, time and place.
3. **Recap** the iteration goal and the stories committed.
4. **Demonstrate** each story from the user's viewpoint (a real scenario, not slides).

5. **Collect feedback:** capture comments as new stories or changes.
6. **Ask the sponsor two questions:** "Is our work to date satisfactory?" and "May we continue?"
7. **Feed the planning:** update the release plan and priorities for the next iteration.

SIGNIFICANCE

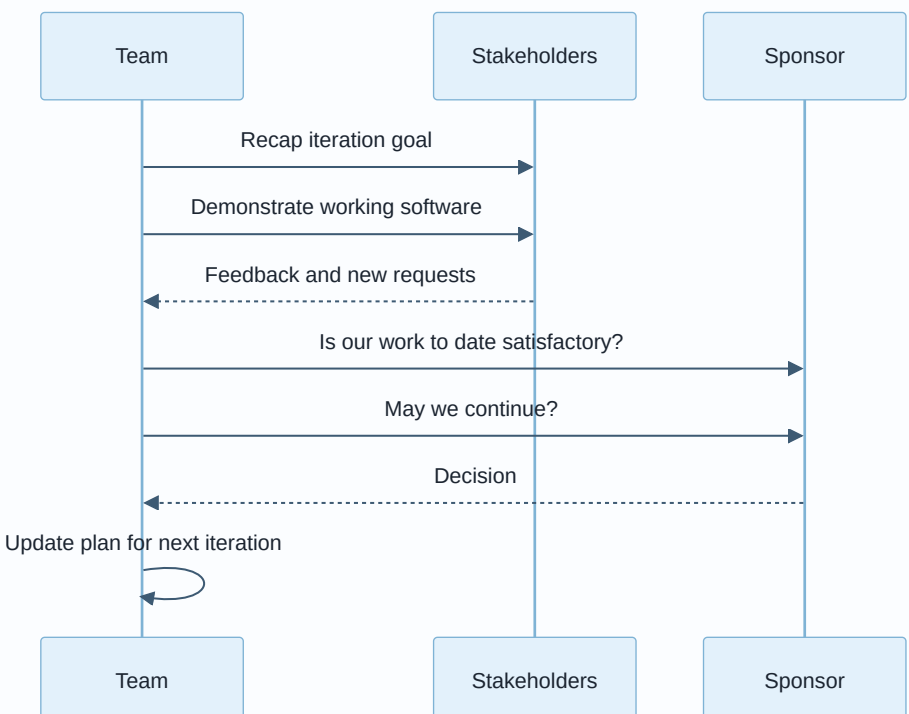
- **Working software:** progress is shown, not claimed.
- **Direct feedback:** confirms the product matches real goals; mistakes surface early.
- **Adapt priorities:** real feedback guides the next iteration.
- **Trust:** weekly proof of delivery builds stakeholder confidence.

EXAMPLE: FOOD DELIVERY APP, ITERATION 3

Team demos "place an order" end to end. The restaurant owner says customers will want to **re-order a past meal**. The team writes a new story, the product manager ranks it high, and it goes into Iteration 4. The sponsor answers "satisfactory" and "yes, continue".

```
Demo checklist
[x] Build deployed to demo environment
[x] Only Done-Done stories shown
[x] Feedback captured as story cards
[x] Sponsor asked: satisfactory? may we continue?
```

DIAGRAM: DEMO SEQUENCE



7 Explain the role of Reporting in Agile. How does effective reporting give visibility into progress, risks and impediments without encouraging misleading performance measurements?

CO3 L2 **10 marks**

Definition: Reporting is sharing honest information about the project's progress and risks with the organization. In XP, reports are a **normal byproduct of the team's work**, so they cost almost no extra effort. Their purpose is to **reassure the organization** that the team is working well.

WHAT THE TEAM SHARES

Report	What it shows
Vision statement	Where the product is going and why.
Weekly product demo	Working software (strongest proof of progress).
Release and iteration plans	What is planned, what is done, forecast dates.
Burn-up chart	Work completed against total scope over time; scope changes are visible.

PRINCIPLES OF EFFECTIVE REPORTING

- **Truthful metrics:** clear progress and risk information.
- **Decision support:** focus on information that helps decisions, not busy counts.
- **Reduce anxiety:** predictable, regular communication builds trust.
- **Surface impediments:** list blockers and risks with the progress, so management can help.

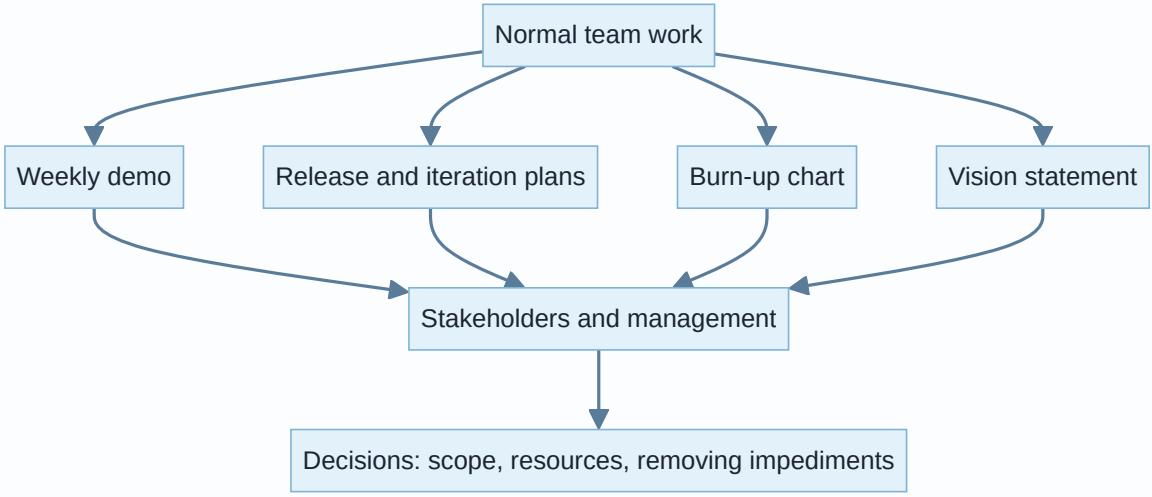
AVOIDING MISLEADING MEASURES

Misleading measure	Why it misleads	Better alternative
Lines of code written	Rewards bloat.	Stories finished (Done Done).
Hours worked	Rewards overtime, not value.	Working software demoed.
"90% complete"	Unverifiable.	Burn-up chart of finished stories.
Velocity as a target	Teams inflate estimates.	Velocity only for forecasting.

EXAMPLE: BURN-UP DATA (STUDENT APP, 40 STORY POINTS)

```
Iteration   Done (cumulative)   Total scope
   1              8                 40
   2             17                 40
   3             25                 46   <- scope grew by 6 (new stories); visible
   4             34                 46
Forecast: 46 points / ~8.5 per iteration = about 6 iterations.
```

DIAGRAM: REPORTING AS A BYPRODUCT



8 **Explain "Done, Done" in Agile. Discuss the activities and quality checks needed before a feature is finished and ready for release.**

`CO3` `L2` `10 marks`

Definition: "Done Done" means a story is **truly finished**: coded, tested, integrated, accepted by the customer and ready to release. A story is not done merely because coding has stopped.

MAIN POINTS

- A shared definition of Done makes progress and velocity meaningful.
- Everything is finished **inside the iteration**; no separate "testing phase" later.
- Nothing is left as hidden work (hidden work becomes late surprises and bugs).

CHECKLIST BEFORE "DONE DONE"

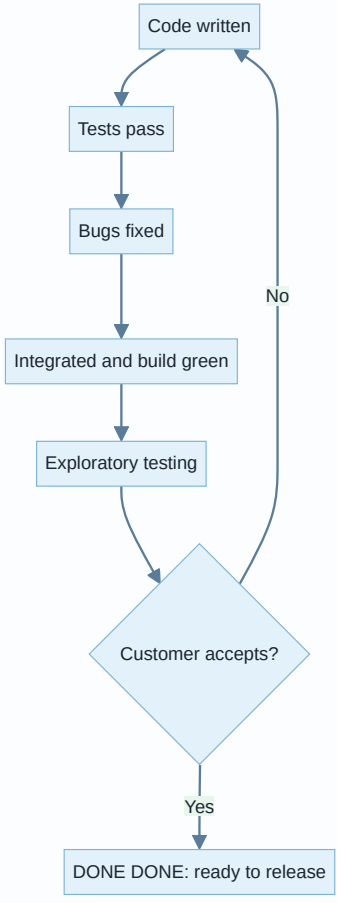
Activity	Quality check
Code complete	Follows coding standards; peer reviewed through pairing.
Tests pass	Unit tests (TDD) and customer tests are green.
Bugs addressed	No known bugs left for this story.
Integrated	Merged to main; ten-minute build and CI pass.
Exploratory testing	Testers probe beyond scripted cases.
Acceptance met	Customer or product manager confirms the story.
Release ready	Can be demoed and deployed with little extra work.

EXAMPLE: LOGIN STORY, STUDENT APP

"Code written" is not enough. The story is Done Done only when the login UI, user table and authentication work, all tests pass, error handling is checked, it is merged and built, and the teacher confirms it at the demo.

```
Definition of Done (team agreement)
[ ] Code written, reviewed (pair), standards followed
[ ] Unit + customer tests pass in CI
[ ] No open bugs on the story
[ ] Merged to main, 10-minute build green
[ ] Exploratory testing done
[ ] Customer accepted
[ ] Deployable to demo environment
```

DIAGRAM: PATH TO DONE DONE



9 **What is Version Control? Explain its working and importance in Agile, including change tracking, collaboration, conflict resolution and recovery of earlier versions.**

CO3 L2 **10 marks**

Definition: Version control is a system (for example Git) that **records every change to files over time**, lets many people work on the same codebase, and allows any earlier version to be restored.

HOW IT WORKS

- **Repository:** central store holding all files and their full history.
- **Clone / pull:** each developer gets a working copy.
- **Commit:** save a snapshot with a message, author and time.
- **Push / merge:** share changes; the system combines work from different people.
- **Branch:** separate line of work for an experiment; merge back when ready.

IMPORTANCE IN AGILE

Need	How version control helps
Change tracking	Who changed what, when and why; compare any two versions.
Collaboration	Many pairs edit in parallel; supports collective ownership and continuous integration.
Conflict resolution	Detects overlapping edits; developers resolve them and re-test.
Recovery	Revert to any earlier version if a change breaks the build.
Releases	Tag versions; rebuild exactly what was shipped.

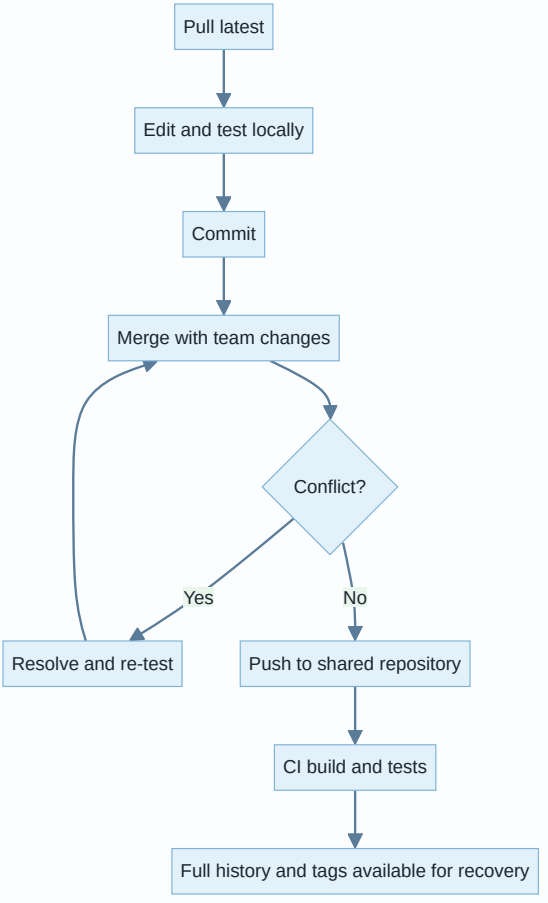
EXAMPLE COMMANDS

```
git pull                              # get latest code
git add Login.py && git commit -m "Add lockout after 3 failures"
git push                              # share (build runs in CI)
git log --oneline                     # change history
git revert a1b2c3d                    # recover: undo a bad commit
git tag v1.0                          # mark a release

# Conflict example: both pairs edited the same line
<<<<<<< HEAD
MAX_TRIES = 3
=======
MAX_TRIES = 5
>>>>>>> feature/lockout
# Resolve by choosing/merging the lines, then re-run tests and commit.
```

Agile practice: commit small and often, integrate several times a day, keep the main branch always buildable (ten-minute build).

DIAGRAM: COMMIT FLOW



10 **Explain Collective Code Ownership in XP. Discuss how shared responsibility for the codebase improves collaboration, reduces dependency on individuals and supports continuous improvement.**

`CO3` `L2` `10 marks`

Definition: In collective code ownership, **the whole team owns all the code**. Any pair may improve or fix any part of the system at any time; no module belongs to one person.

BENEFITS

- **Collaboration:** no waiting for "the owner"; pairs rotate across the code.
- **Less dependency on individuals:** if one person leaves or is absent, others already know the code (higher "bus factor").
- **Faster delivery:** bugs and features are handled by whoever is free.
- **Continuous improvement:** anyone who sees messy code can refactor it immediately.
- **Better quality:** many eyes, consistent style, shared design knowledge.

WHAT MAKES IT SAFE (SUPPORTING PRACTICES)

Practice	Role
Version control	Every change is tracked and reversible.
TDD and automated tests	Safety net: wrong changes fail quickly.
Continuous integration	Frequent merging; broken build is caught within minutes.
Coding standards	Code reads as if one author wrote it.
Pair programming and rotation	Knowledge spreads across the team.

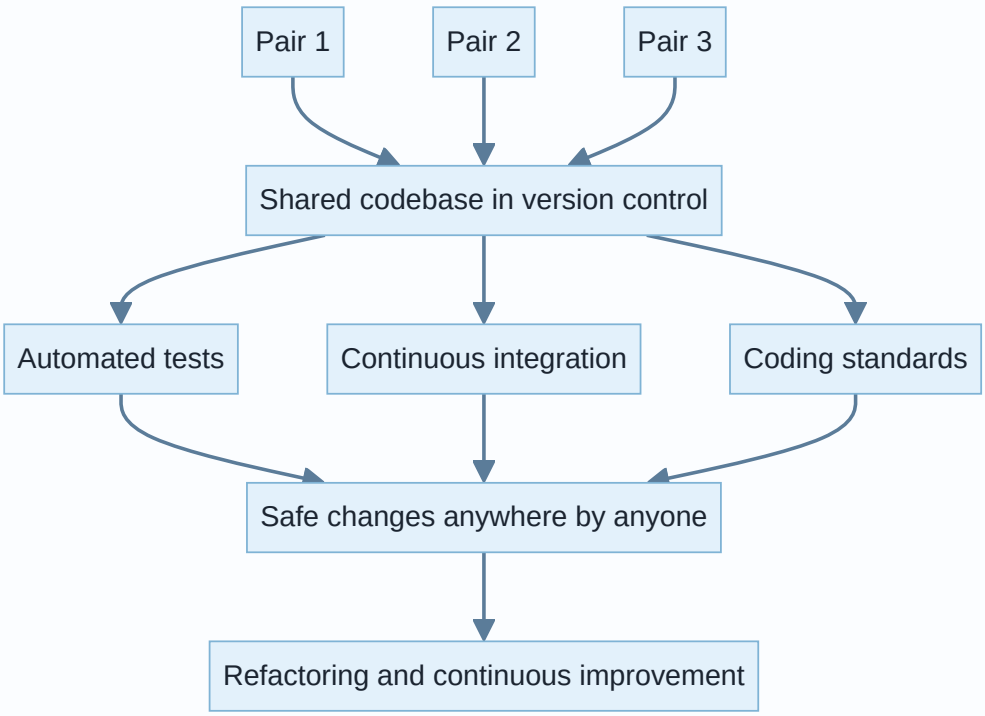
EXAMPLE: BANKING APP

Rahul wrote the `Transfer` module. While working on statements, Priya's pair finds that `Transfer` rounds amounts wrongly. They fix it at once, add a test and push. No ticket to Rahul, no delay. When Rahul is on leave, the team still ships.

```
Owner model        : bug in Transfer -> wait for Rahul -> blocked 3 days
Collective model   : bug in Transfer -> any pair fixes + test -> merged same day

def test_transfer_rounds_to_paise():       # added by Priya's pair
    assert transfer(100.005, "A", "B").amount == 100.01
```

DIAGRAM: SHARED OWNERSHIP



Primary reference: James Shore and Shane Warden, *The Art of Agile Development*, Part II: Practicing XP. Answers written for the 10-mark pattern: definition, points, example, diagram.